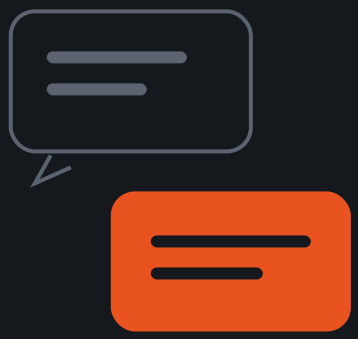
HUF / AI PLATFORM / PRODUCT GUIDE

WhatsApp in Huf: what you can build

How Huf turns WhatsApp into an AI-agent channel

An overview for buyers, operators and developers.



Huf pre-develop, October 2026

Contents

01 **Executive overview** 3
What you can do with WhatsApp in Huf, at a glance 4
Who this guide is for 4

02 **How it works: a message's journey** 5
Step by step 5
What is synchronous and what is queued 6

03 **Get connected in minutes** 7
What you need 7
The setup checklist 8
Token health and the webhook 9
Several numbers, fully separated 10

04 **Who can talk to your agent** 12
Direct-message policy 12
Approving people 12
STOP and START 13
Run-as user and permissions 13

05 **Everything customers can send** 15
Media safety 16

06 **Voice and audio in depth** 17
Hearing voice notes 17
Speaking back 17
Safe by default 17
Voice-first, multilingual 18

07 **Rich outbound messages** 19
What can be sent 19
Link preflight and uploads 19
Typing and read receipts 19
The send panel 20

08 **Templates in depth** 22
The template studio 22
Create in three steps 24
Rules enforced before Meta sees it 25
Sending with parameters 25
Status, quality and the edit quota 26
The template guard 27
Drafts and human approval 27

09 **The 24-hour window and outbound policy** 29
Window rules 29
Fallback template 29
What an agent may send 29
Idempotent sends 30
Account state 30

10 **Messages, delivery and drafts** 31
The message log 31
Detail sheet 32
Events and observability 33

11 **Agents, tools, flows and automations** 34
The agent's WhatsApp tool 34
Routing to the right agent 34
Flows: deterministic on WhatsApp 34
Approvals 35

12 **Reliability and security** 36
Reliability 36
Failures are never silent, never leaky 36
Security model 36
Tested against a strict simulator 37

13 **Eight real use cases** 38
Use case 1: Bakery order-taking and order status 39
Use case 2: Appointment booking and reminders 41
Use case 3: Support triage with pairing approval and a ticket 43
Use case 4: Internal ops bot 45
Use case 5: Field reporting: photo, location and voice note 47
Use case 6: Lead qualification from ad clicks 49
Use case 7: Collections and payment reminders 51
Use case 8: Voice-first customer in another language 53

14 **Why Huf is a very strong WhatsApp solution** 55
Feature coverage matrix 56

15 **Honest limitations and roadmap** 57

16 **Appendix** 58
Glossary 58
Meta error codes Huf explains in plain language 58
Setup checklist 59
Where the code lives 60

01

Executive overview

Huf connects an AI agent, or a deterministic workflow, to a WhatsApp Business number through Meta's official Cloud API, and treats the hard parts (verification, policy, reliability, permissions) as part of the product.

WhatsApp is where many customers already are. The difficult part of serving them with AI is not generating a reply. It is everything around it: proving the message really came from Meta, never answering the same message twice, respecting STOP requests and the 24-hour rule, understanding voice notes and photos, and making sure the agent can only touch the data it is allowed to touch.

Huf handles that whole path. A customer message is signature-checked, de-duplicated, admitted by an access policy, run through an agent under a least-privilege user, and answered in WhatsApp-native formatting. Failures are recorded and surfaced, never silently dropped. An operator console at /whatsapp shows health, token status, templates, a message log with delivery ticks, and an access list.

> **THE CORE PROMISE, IN CODE**
>
> Automation that does not lose, duplicate or leak messages. This is a WhatsApp Business Platform (Cloud API) integration, not a WhatsApp Web or personal-number workaround.

20	15+	~1,070	24 h
actions in the single agent WhatsApp tool	inbound message kinds with defined behaviour	WhatsApp tests, run against a strict Graph API simulator	window rule enforced, with template fallback

What you can do with WhatsApp in Huf, at a glance

Understand voice notes
Speech-to-text turns a voice note into a transcript the agent reads.

Read photos and documents
Images go to the model as vision input; PDFs and Office files are text-extracted.

Reply like WhatsApp
Markdown becomes WhatsApp formatting; long answers split safely at 4,096 characters.

Answer with voice
Optional voice-note replies via the agent's own TTS, with automatic fallback to text.

Send rich messages
Images, documents, video, audio, location, contacts, reply buttons, lists, link buttons.

Manage templates
Wizard with live preview, drafts for approval, status and quality tracking.

Obey the 24-hour window
Free-form inside the window, approved fallback template outside it.

Control who can talk
Allow list, pairing approvals, open or disabled; STOP and START honoured.

Run with least privilege
The agent runs as a named Frappe user, so ERP permissions apply.

Choose deterministic flows
Route a number to a Flow with text ops, per-sender state and permission-checked doc creation.

Never fail silently
Idempotency, ordering, retries, a reaper, dead letters and a plain-language apology.

See what happened
Health banner, 24-hour activity, message log with sent, delivered, read, played, failed.

Who this guide is for

If you are a...	Read first	You will learn
Buyer or business owner	Chapters 1, 13, 14, 15	What Huf can do on WhatsApp, eight concrete use cases, where it is strongest and where it is not.
Operator or support lead	Chapters 3, 4, 8, 9, 10	How to connect a number, approve people, run templates and read the message log.
Developer or integrator	Chapters 2, 11, 12, 16	Message pipeline, the agent tool, flows, reliability model and where the code lives.

A WORD ON EVIDENCE

Every capability in this guide is backed by code and tests in the Huf pre-develop branch. Where something is partial, experimental or not built, the guide says so (chapter 15). Screenshots come from the real Huf console with fictional demo data; mock-ups are labelled as illustrations.

02

How it works: a message's journey

From the customer's phone to your agent and back, with every check along the way.

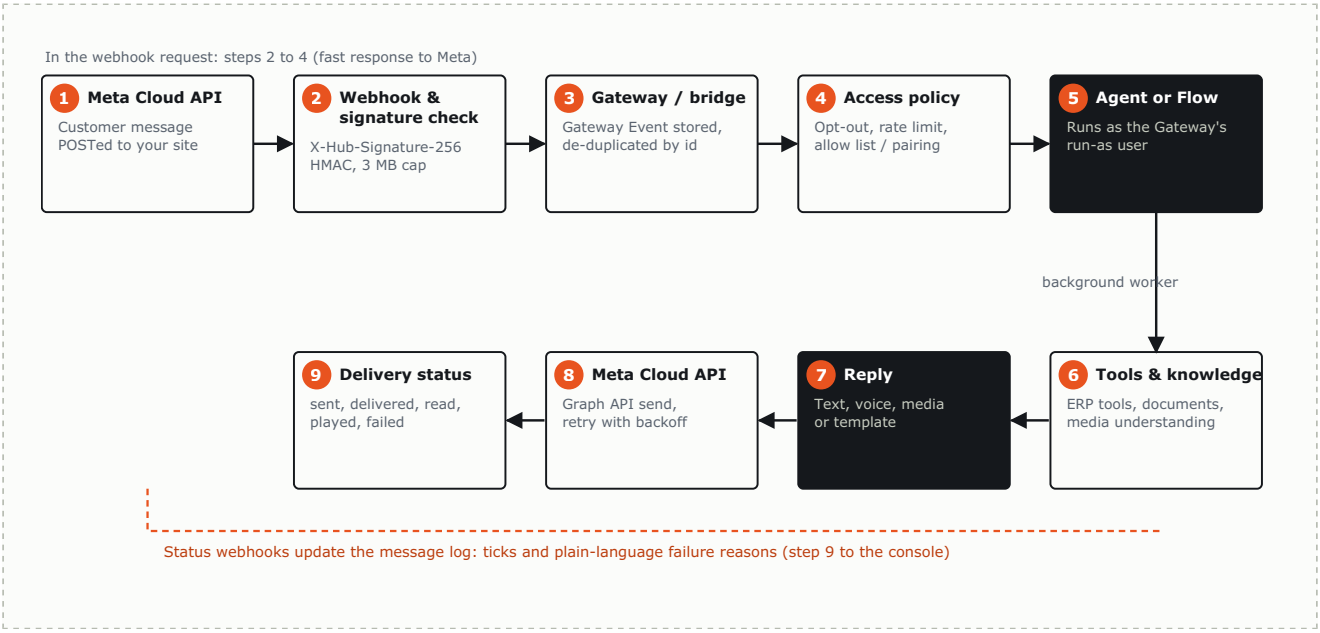


Figure 1. A message's journey through Huf. Steps 2 to 4 happen synchronously inside the webhook request; steps 5 to 8 run in a background worker. Illustration, not a screenshot.

Step by step

1. **Meta delivers.** The customer writes to your number. Meta POSTs to one callback URL per Gateway (one Gateway per phone number).
2. **Verified before stored.** The webhook checks the HMAC-SHA256 signature over the raw body using your Meta app secret, in constant time. Unsigned or wrongly signed traffic gets HTTP 401 and counts against a rate budget of 100 requests per 60 seconds per gateway.
3. **One event per message.** Each message becomes a Gateway Event with a unique key derived from the gateway and Meta's message id. A redelivered webhook returns duplicate and creates nothing.
4. **Admission.** Reactions, edits, revokes and system messages are recorded without running an agent. Then the per-sender rate limit (default 20 per 10 minutes), STOP/START handling, and the access policy (Allow list, Pairing, Open, Disabled).
5. **Routing.** The first enabled Gateway Binding by priority wins; otherwise the Gateway's default Agent or Flow. If nothing matches, the event is Unrouted and no model is called.
6. **Prepare and run.** In a background worker Huf switches to the run-as user, downloads media (images to vision input, audio to a transcript, documents to extracted text), marks the message read, shows the typing indicator, and runs the agent under a per-conversation lock so one sender's messages are handled in order.
7. **Reply.** Markdown is converted to WhatsApp formatting and split into parts of at most 4,096 characters. The 24-hour window is checked; if it is closed, an approved fallback template is used. Transient errors are retried with backoff. Optionally the reply goes out as a voice note.

8. **Delivery status returns.** Meta posts sent, delivered, read, played or failed. Huf maps them onto the originating event and the console's message log.

> **WHERE THE OPTIONAL FRAPPE_WHATSAPP CONNECTOR FITS**
>
> In Direct mode, Huf core calls the Graph API itself. In Bridge mode (the optional frappe_whatsapp app installed and linked to the Gateway) replies are also created as Outgoing WhatsApp Message rows, inbound messages are mirrored for history, and the Templates and Messages console tabs work. Without the connector, receiving, agent runs, replies, template fallback, media, voice, STOP handling and health still work; template management and message history do not.

What is synchronous and what is queued

Runs inside the webhook request	Runs in a background worker
Signature check, payload parse, de-duplication, event insert, admission, routing, enqueue	Media download and preparation, the agent run, reply sending and retries, the 10-minute reaper, call answering

The Graph API version defaults to v25.0 and can be pinned per gateway. Because everything slow happens after the response, Meta gets a fast acknowledgement and is far less likely to redeliver.

03

Get connected in minutes

One Gateway per WhatsApp number, four credentials, one callback URL, and a checklist that tells you what is left.

What you need

Credential	Where it comes from	Notes
phone_number_id	Meta WhatsApp product	Identifies the number; two enabled gateways cannot share one.
access_token	Permanent system-user token in Meta Business settings	Stored in a Password field; never shown again.
webhook_verify_token	You choose it	Compared in constant time on Meta's GET challenge.
app_secret	Meta app settings	Used to verify the signature of every POST.

The adapter refuses to start if any of the four is missing. The console shows only whether each is saved, never the value.

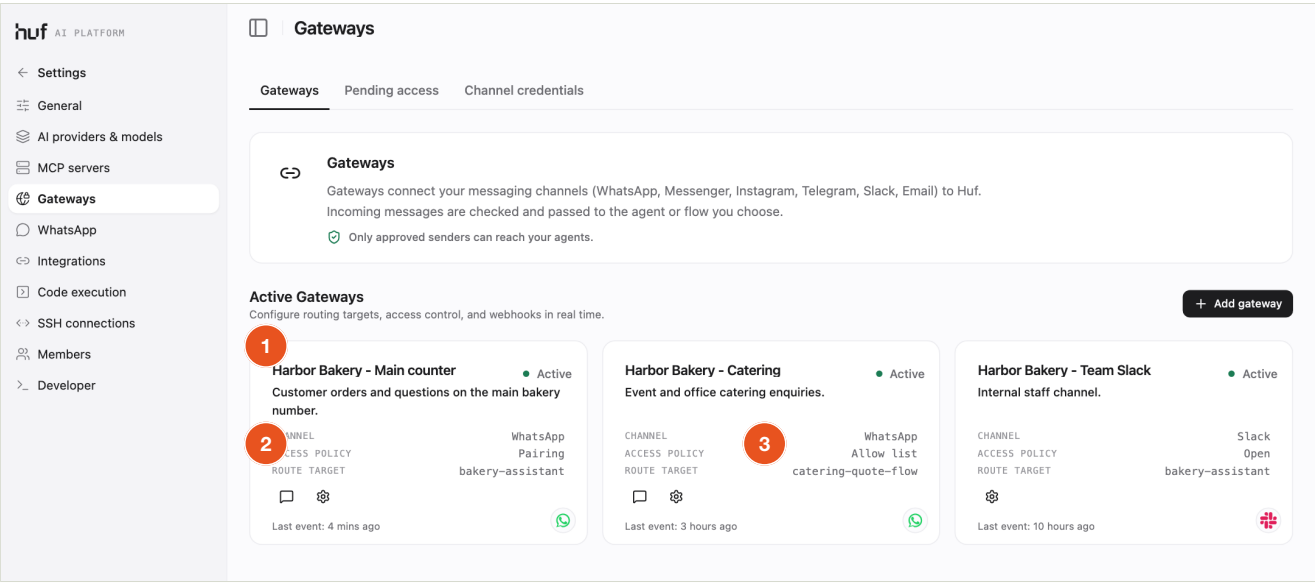


Figure 2. The Gateways page lists each connected number with its channel, access policy and route target. Screenshot of the Huf console, fictional Harbor Bakery data.

1. One card per number: two WhatsApp gateways and one Slack gateway.
2. Access policy per gateway (Pairing, Allow list, Open).
3. Route target: an agent or a flow.

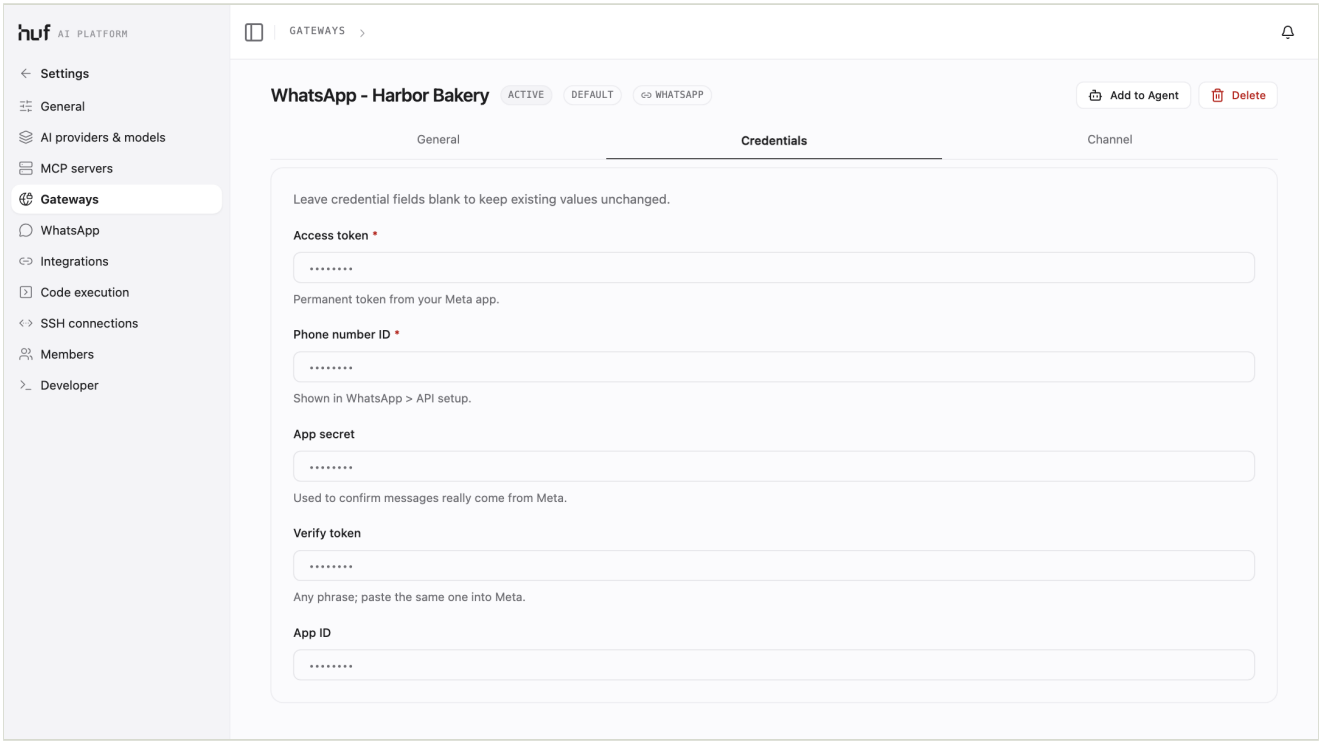


Figure 3. The gateway's Credentials tab. Values are hidden; secrets live in Password fields. Screenshot of the Huf console, fictional Harbor Bakery data.

The setup checklist

The Overview tab carries a guided seven-step checklist: create a Meta app, add credentials, set the callback URL, subscribe to messages, add the app secret, turn the number on, and send a test message. It is a checklist on the page rather than a modal wizard; steps deep-link into Settings. Because Meta has no read API for webhook state, the callback and subscription steps are inferred from what Huf has seen (a verification request, events received).

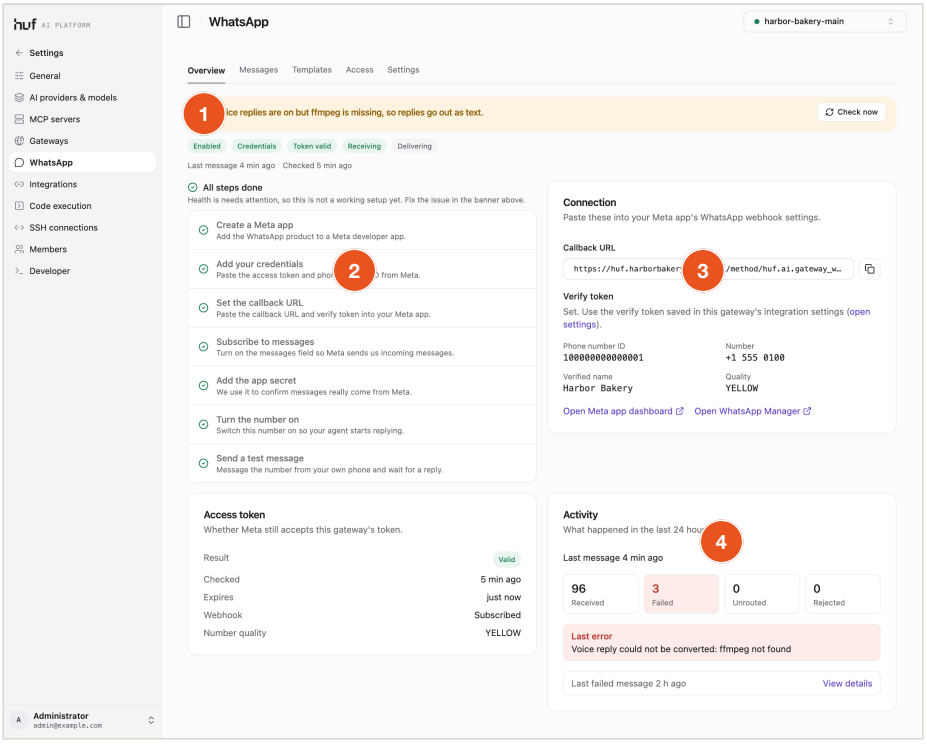


Figure 4. The Overview tab with the full checklist, connection card, token card and 24-hour activity. Here a voice-reply warning is showing. Screenshot of the Huf console, fictional Harbor Bakery data.

1. Health banner in plain language, with a Check now button.
2. Seven-step checklist with deep links.
3. Callback URL and verify token to paste into Meta.
4. 24-hour activity: received, failed, unrouted, rejected, and the last error.

Token health and the webhook

Huf asks Meta whether the token is still accepted and whether the app is subscribed. It shows Valid or Not valid, the expiry date (Never for permanent tokens), missing permissions and subscription state. Results are cached for 10 minutes, live checks are limited to one per 10 seconds, and Check now forces a fresh read. A rejected token raises an error-level health reason and blocks the credentials step.

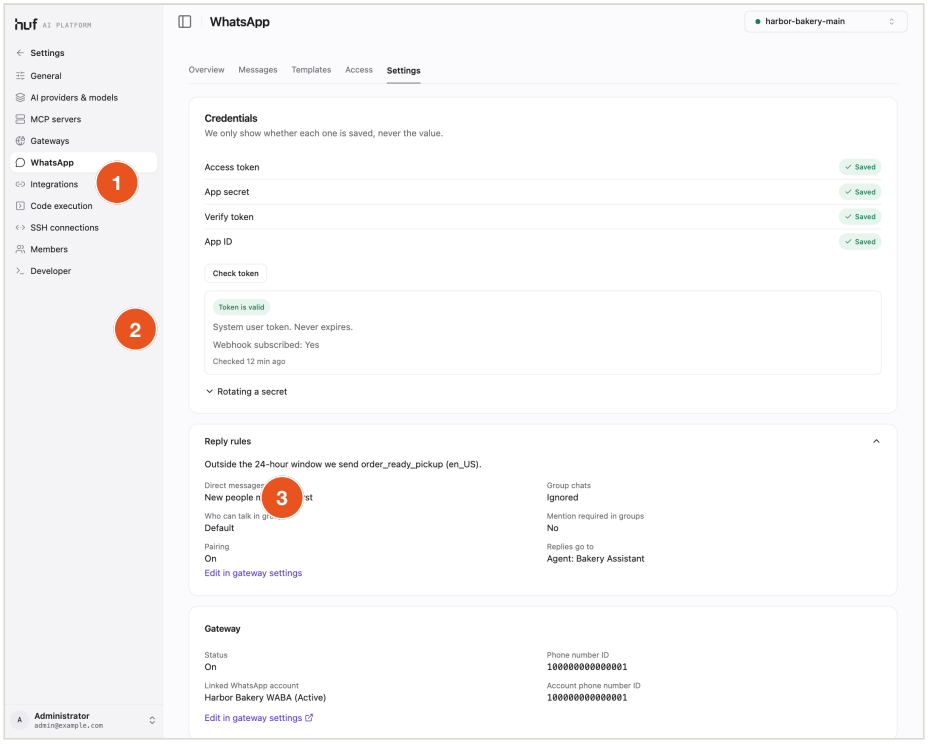


Figure 5. The Settings tab: credential status, token check, reply rules and the linked gateway. Screenshot of the Huf console, fictional Harbor Bakery data.

1. Credentials: saved or not, never the value.
2. Token is valid. System user token, never expires, webhook subscribed.
3. Reply rules in plain words: fallback template outside the window, who can message, where replies go.

> **ROTATE SECRETS IN TWO PLACES**
>
> In Bridge mode, refresh both the gateway credential and the linked WhatsApp Account token when you replace a system-user token.

Several numbers, fully separated

Each phone number is its own Gateway with its own credentials, callback URL, route and policies. Two enabled WhatsApp gateways cannot share a phone_number_id. Events, the 24-hour window, conversations, replies, the message log and template state are scoped per gateway even when the same customer writes to several numbers. A signature valid for gateway A is rejected on gateway B, and a signed payload carrying an unknown phone_number_id is ignored and recorded as an error. In the console, the gateway switcher (top right, kept in the URL) changes which number you are looking at.

> **TESTED ISOLATION**
>
> test_whatsapp_multi_gateway_isolation covers: message log scoped per gateway, search that cannot reach the other gateway, per-gateway setup status, and a failed signature on one gateway leaving the other untouched.

SETUP IS GUIDED, NOT AUTOMATED

There is no embedded signup. The operator creates the Meta app, the system-user token, the webhook and the subscriptions by hand; the checklist guides but does not do it for you. Meta's per-number callback override is documented in Huf's setup notes but its request shapes are marked to verify before use.

04

Who can talk to your agent

Admission policy, pairing approvals, STOP/START and the identity the agent runs as.

Direct-message policy

Policy	What happens to a new sender
Allow list (default)	Only an approved Gateway Access Entry may talk to the agent.
Pairing	An unknown sender creates a pending request with a one-time code (format PAIR-XXXXXXXX). The first message is not executed. Staff approve in the console.
Open	Anyone may talk to the agent. The per-sender rate limit still applies.
Disabled	No direct messages are admitted.

Pairing codes use an 8-character alphabet that skips look-alike characters (I, L, O, 0, 1). Approval attempts are rate-limited (5 failures per minute), pending requests are capped (default 50), pairing replies have a daily budget (default 20), and pending entries expire. A sender is also capped at 3 pairing requests per day (expired or revoked rows still count), so one number cannot flood the list with new pairing rows.

Approving people

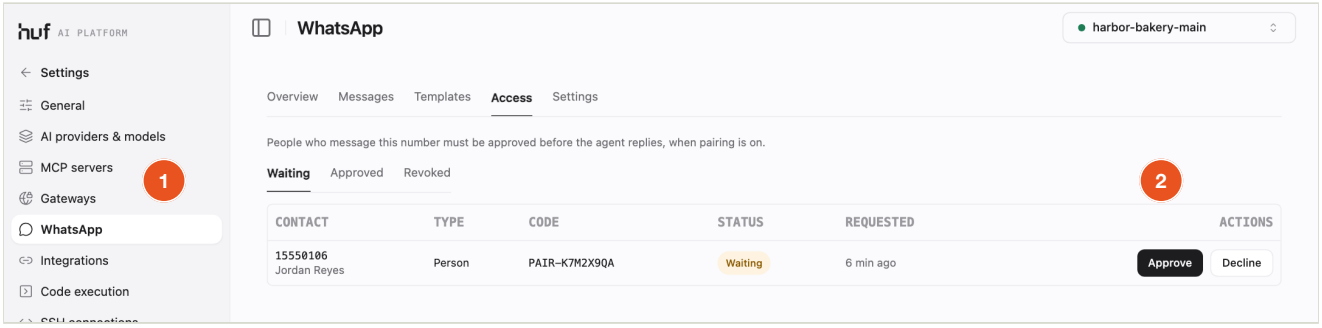


Figure 6. The Access tab, Waiting list: one pending pairing request with Approve and Decline. Screenshot of the Huf console, fictional Harbor Bakery data.

1. Contact and pairing code.
2. Approve or Decline.

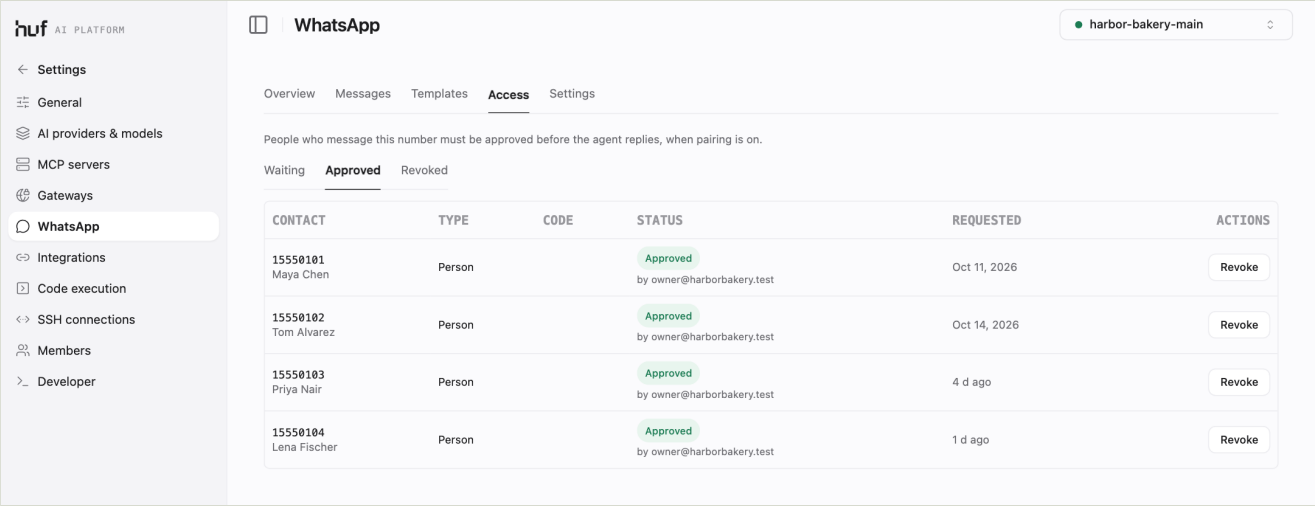


Figure 7. Approved senders. Entries can be revoked at any time. Screenshot of the Huf console, fictional Harbor Bakery data.

The Access tab has Waiting, Approved and Revoked lists showing the contact, code, status and request time. Approval restores admission; the next message is processed normally.

STOP and START

Opt-out is handled before any agent runs. The defaults are STOP, UNSUBSCRIBE, CANCEL, END, QUIT to opt out and START, UNSTOP, SUBSCRIBE to resubscribe. A STOP from an admitted sender records the opt-out, sends a confirmation and starts no run. Later messages from an opted-out sender are rejected, and every outbound path checks the opt-out before sending, including templates and the agent tool. Keyword lists and the confirmation text are editable per gateway. A STOP from an unknown sender is ignored: no row, no reply.

Meta's own marketing opt-out preferences are also stored: marketing-category templates to a contact who has stopped marketing are refused.

> **META PREFERENCE PAYLOAD**
>
> The shape of Meta's user_preferences payload is marked unverified against live Meta in the code. Huf's own STOP/START handling is fully tested.

Run-as user and permissions

The agent never runs as Guest. Each Gateway has a Run as user, which must hold an allowed role and the agent.use capability and have access to the default agent; saving an enabled Gateway validates this. Everything the agent reads or writes is under that user's Frappe permissions. The setup guidance is explicit: never Administrator.

> **ERP-GRADE PERMISSIONS**
>
> A bakery bot can run as a Bakery Staff user who can read Items but not Salary Slips. The agent has no way to see data the user cannot see, because it goes through the same permission checks as a person at a desk.

Frappe permissions are role-based, per doctype, and can be narrowed by user permissions. That is what makes a WhatsApp agent safe to give real business tools.

The per-sender inbound limit (default 20 messages per 10 minutes) rejects bursts before consent or pairing logic, so a flood from one number cannot reach the agent.

> **GROUPS ARE NOT SUPPORTED ON WHATSAPP**
>
> Room and group policy fields exist on the Gateway for other channels, but the WhatsApp adapter has no group handling. The Settings tab shows group chats as ignored.

05

Everything customers can send

Each inbound type has a defined behaviour. The adapter converts it into text the agent can read and keeps the raw message on the event.

Customer sends	What Huf does	Limits
Text	Body becomes the prompt.	16,000 characters on the event
Voice note / audio	Saved as a private file, transcribed with the agent's speech-to-text, transcript prefixed [Voice note transcript]. If transcription fails the agent is told so.	Transcript capped at 8,000 characters
Image	Passed to the model as image input; the caption is the text.	5 MB
Document	Text extracted (PDF via OCR with fallbacks, Office and text locally), labelled and appended.	100 MB; 12k chars per document, 24k total
Video	Saved and listed as an attachment. Whether the agent can see video content is not confirmed.	16 MB
Sticker	Accepted as media.	500 KB
Location	Rendered as [Location: lat,lng name address] and starts a run.	
Contacts	Up to 5 contact cards, 3 phones each, rendered as [Contact: ...].	
Buttons and lists	The reply's title (or id) becomes the text.	
Quick-reply (template button)	Button text or payload becomes the text.	
WhatsApp Flow reply	[Form response: name] plus compact JSON.	2 KB
Catalog order	[Order: 2 items, catalog X] with SKU, quantity, price.	10 items
Product inquiry	[Product: title (retailer_id)].	
Ad click (referral)	Referral fields (source, headline, ctwa_clid) kept on the event; headline prefixed as [Ad referral: headline].	
Reaction	Recorded only. No run, no reply, and it never opens the 24-hour window.	
Edit / delete	Recorded only. An edit marks the original superseded; a delete marks it deleted and rejects it if not yet processed, so the agent never answers a deleted message. Only the same sender can affect the original.	
System message	Recorded only.	
Unsupported type	Fixed reply: "Sorry, I can't read this type of message. Please send text, a photo, a document, audio or a location." No LLM call; silent for Meta code 131060; never sent to unadmitted senders.	

Media safety

Downloads are guarded by per-kind size limits checked on both declared and streamed size, a MIME allow-list, rejection of SVG, HTML and executables, and filename sanitising. If a download fails the agent is told the attachment is unavailable, and the customer gets a localised could-not-open-the-file message if the agent says nothing.

ECHOES AND BSUIDS

Messages the business itself sent are dropped, not ingested as customer messages. A sender identified only by a business-scoped user id (BSUID) is ingested and the id stored, but Huf cannot answer it: outbound to a BSUID is refused because Meta needs a phone number.

FORWARDED AND QUOTED CONTEXT

The forwarded flag and quoted message id are stored on the event, but nothing in the code shows them to the agent.

06

Voice and audio in depth

Voice notes in, optional voice notes out, in the customer's own language.

Hearing voice notes

A voice note is downloaded, saved as a private file and transcribed with the speech-to-text model configured on the agent (the stt_model field). The transcript is prepended to the prompt as [Voice note transcript (name)]: ... so the agent simply reads it. If transcription fails, the agent is told the voice note could not be transcribed rather than being left to guess.

Speaking back

Voice replies are a per-Gateway setting with three modes. They apply only when the customer sent a voice note.

Mode	Behaviour
Off (default)	Replies are text.
Voice only	One voice note replaces the text reply.
Text and voice	Both a text message and a voice note are sent.

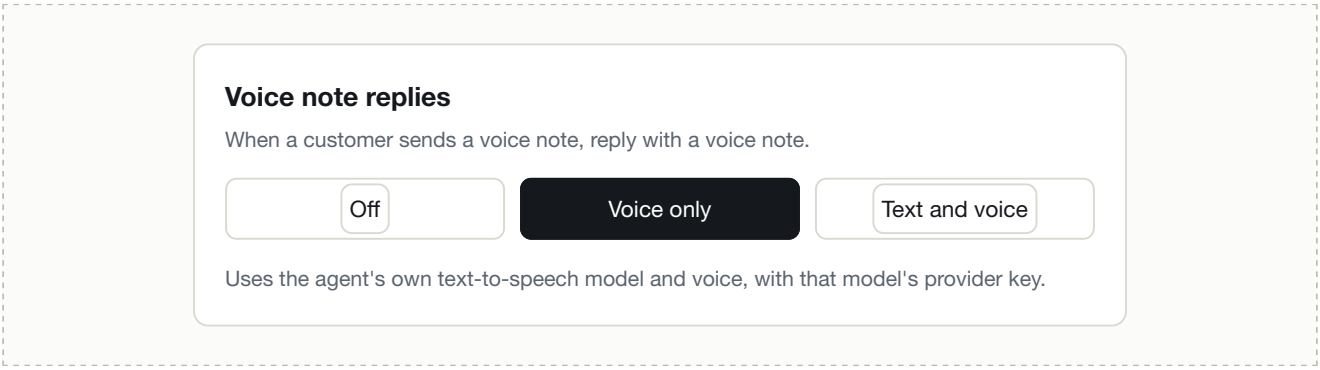


Figure 8. A settings-card illustration of the three voice reply modes (the Gateway form field is voice_reply_mode). Illustration, not a screenshot.

The reply is synthesised by the agent's own TTS model and voice (tts_model, tts_voice) using that model's provider key, then converted to Ogg/Opus mono by ffmpeg and sent as a voice note. Replies longer than 1,500 characters (markdown stripped) go out as text. All the usual gates still apply: STOP, the 24-hour window and BSUID refusal.

Safe by default

- **Never lost.** Any failure (TTS error, ffmpeg missing, bad audio) falls back to the text reply.
- **Never doubled.** A replayed event does not send a second voice note.

- **Validated.** Output is mono Opus at 32 kbps, and uploads are checked for Ogg and Opus headers before sending.

> **FFMPEG WITH LIBOPUS IS REQUIRED ON EVERY WORKER HOST**
>
> When voice replies are on and ffmpeg is missing, the health banner shows: Voice replies are on but ffmpeg is missing, so replies go out as text. Saving the Gateway shows the same warning. The check runs on the site host only; verify worker hosts yourself.

Voice-first, multilingual

Because the agent reads a transcript and the TTS voice is the agent's own, a customer who speaks Spanish can get a Spanish voice note back if the agent's instructions say to reply in the customer's language. Failure apologies are also localised: English, Spanish, French, Portuguese, German and Arabic, chosen from the contact's language, then the gateway's template language, then English.



Figure 9. A voice-only gateway: voice notes get voice notes, text gets text. The italic line is the transcript the agent works from. Illustrated conversation with fictional people and numbers; not a screenshot.

07

Rich outbound messages

Everything WhatsApp can carry, from the console or from the agent, validated before it reaches Meta.

What can be sent

Kind	Details
Text	Markdown from the model becomes **bold**, _italic_, ~strike~, quotes and code spans; tables and headings are flattened; long replies split at paragraph, line, sentence or word boundaries into parts up to 4,096 characters; the first part quotes the customer's message; optional link previews.
Image, video, audio, document	By public https link or by uploaded media id. Documents carry a filename and caption.
Voice note	Ogg/Opus, converted automatically when needed.
Sticker	Supported by the builders and console.
Location and contact cards	Validated payloads.
Reactions	Add or remove, emoji validated.
Reply buttons	Up to 3 buttons.
List menus	Up to 10 rows.
Call-to-action URL button; location request	Supported by the payload builders and agent tool.

Payload builders validate against Meta's limits and return every problem at once instead of one failure per attempt. Fenced code blocks are never cut mid-fence unless huge.

Link preflight and uploads

Meta accepts a dead media link and then silently never delivers. Huf prevents that: before posting a link it fetches it (https only, up to 3 redirects, every hop checked against server-side request forgery, short timeout) and requires HTTP 200, a matching content type (never a web page) and Meta's size cap. Failures come back as media_link_unreachable, media_link_wrong_type or media_link_too_large. Uploading a file skips the link fetch; the agent can send a Frappe File after an access check that the run may use it.

Typing and read receipts

Before the run, Huf marks the customer's message read and shows the typing indicator (valid up to 25 seconds). It is best effort and never blocks the reply; you can disable it per gateway.

The send panel

Operators with write authority on the Gateway can send any supported type from the Messages tab. Text only goes inside the 24-hour window; a template can go at any time.

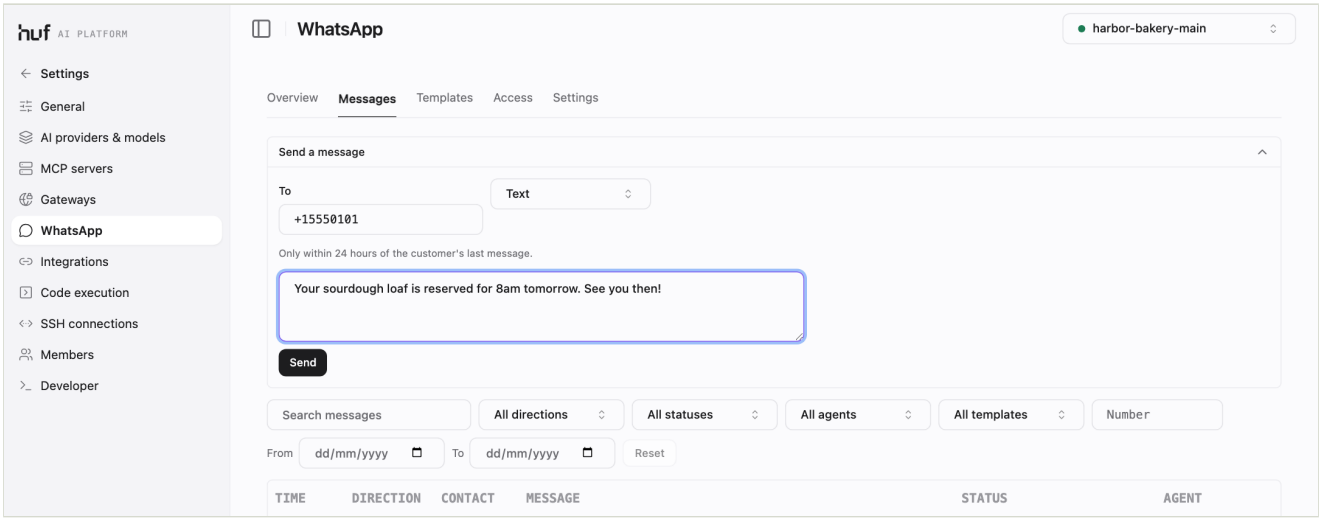


Figure 10. Send panel: plain text, with the window note. Screenshot of the Huf console, fictional Harbor Bakery data.

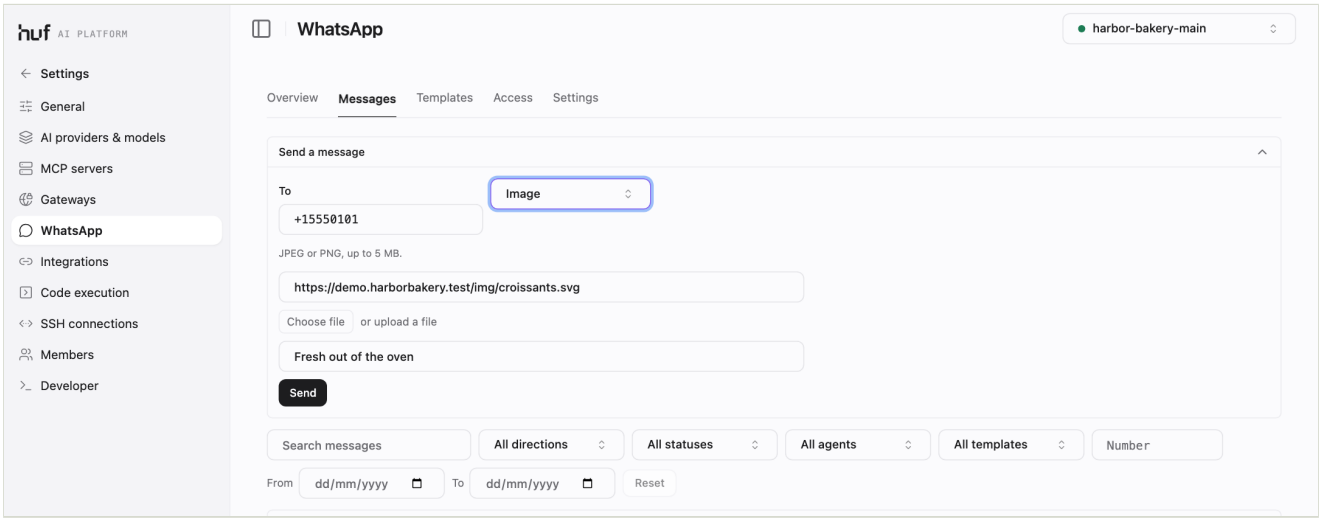


Figure 11. Send panel: an image by public link plus caption. The link is preflight-checked. Screenshot of the Huf console, fictional Harbor Bakery data.

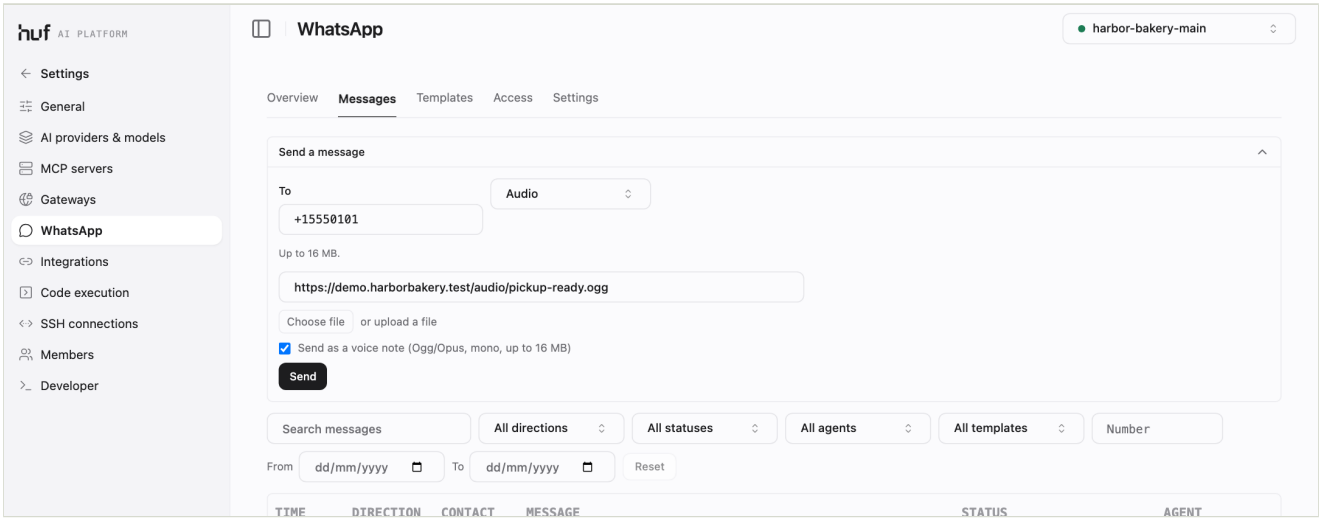


Figure 12. Send panel: audio by link with Send as a voice note checked. Screenshot of the Huf console, fictional Harbor Bakery data.

FLOW MESSAGE BUILDER

A builder for WhatsApp Flow messages exists in code but is not exposed by any tool action or console form, so it is not offered here. The agent's reply pipeline also does not attach media automatically; media goes out through explicit tool actions or the console.

08

Templates in depth

Pre-approved messages are how a business starts or resumes a conversation. Huf treats the whole lifecycle as a first-class feature.

The template studio

The Templates tab lists every template with status badges (Approved, In review, Rejected, Paused) and quality badges, in each language.

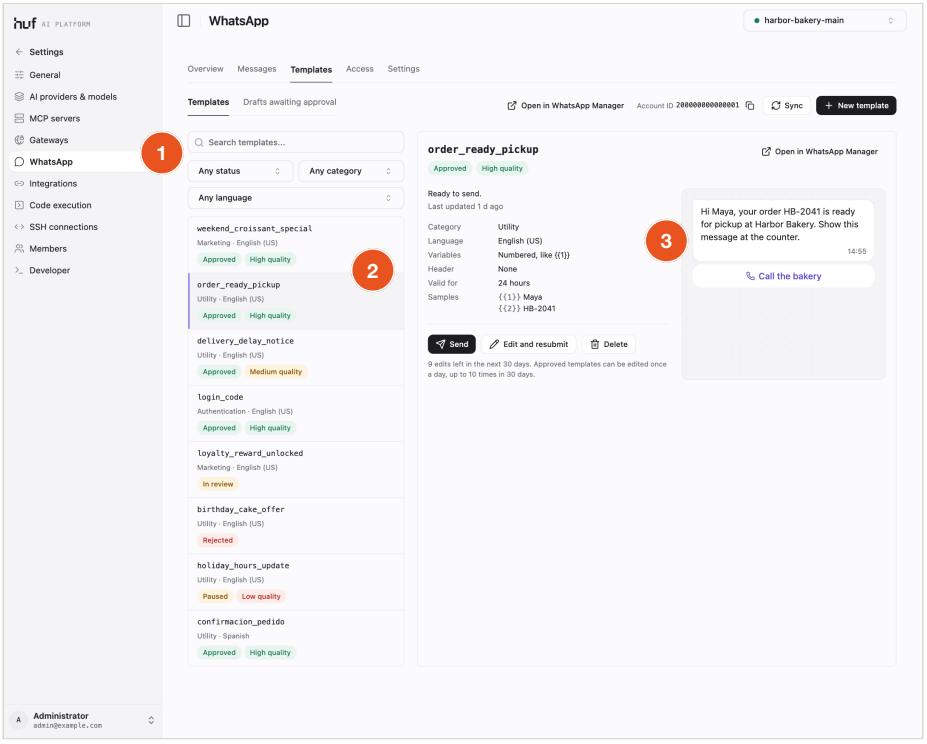


Figure 13. Template list with status and quality badges; Marketing and Utility templates; Authentication templates synced from Meta are shown read-only; English and Spanish. Screenshot of the Huf console, fictional Harbor Bakery data.

1. Search and filters.
2. Status and quality badges per template.
3. Detail pane for the selected template.

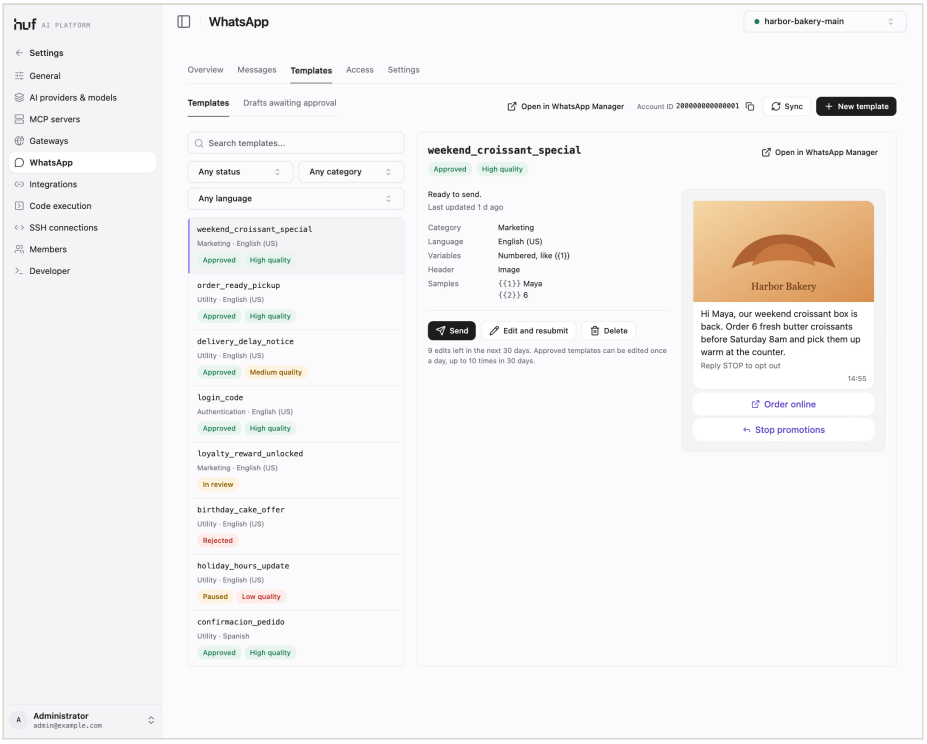


Figure 14. Detail pane with a live phone preview: image header, body variables, footer, URL and quick-reply buttons. Screenshot of the Huf console, fictional Harbor Bakery data.

Create in three steps

The wizard has three steps: Pick a type, Write it, Check and send, with a live WhatsApp-style preview. Marketing and Utility are available. Creating Authentication templates is rejected (one-time-password templates use a fixed WhatsApp format and are not available here yet); Authentication templates synced from Meta appear read-only in the list; Catalogue, Flows and Calling kinds are listed as not available.

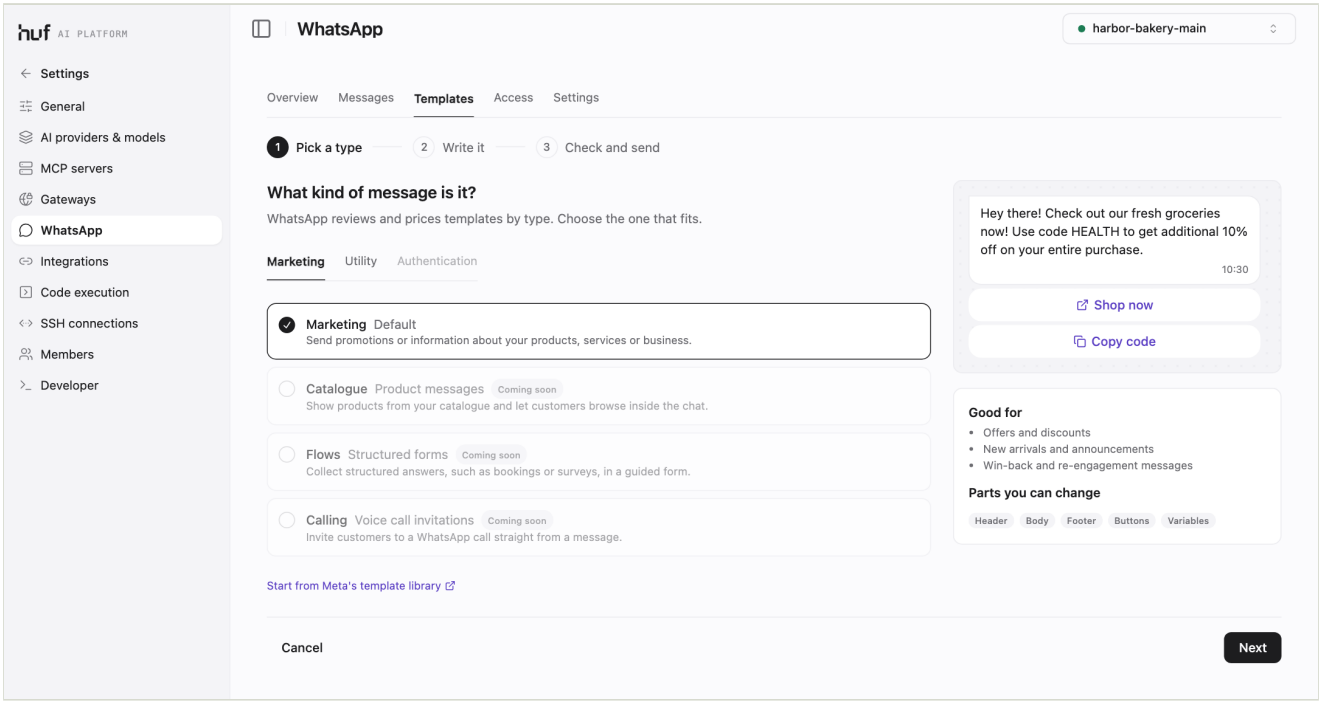


Figure 15. Step 1: pick a type. Screenshot of the Huf console, fictional Harbor Bakery data.

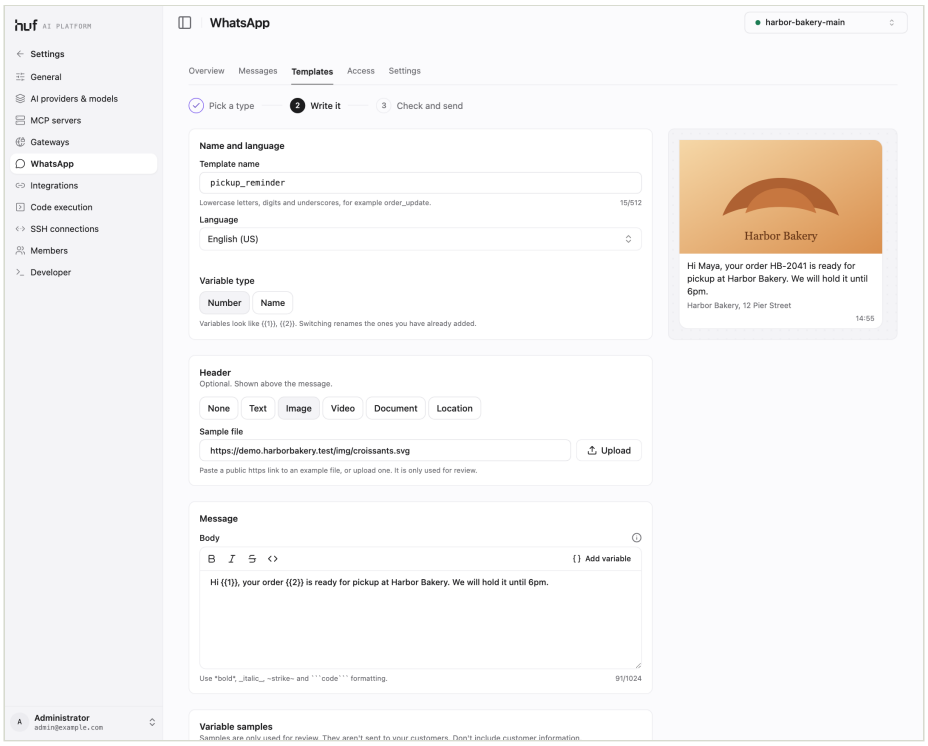


Figure 16. Step 2: name, variable type, image header, body, sample values, with the live preview. Screenshot of the Huf console, fictional Harbor Bakery data.

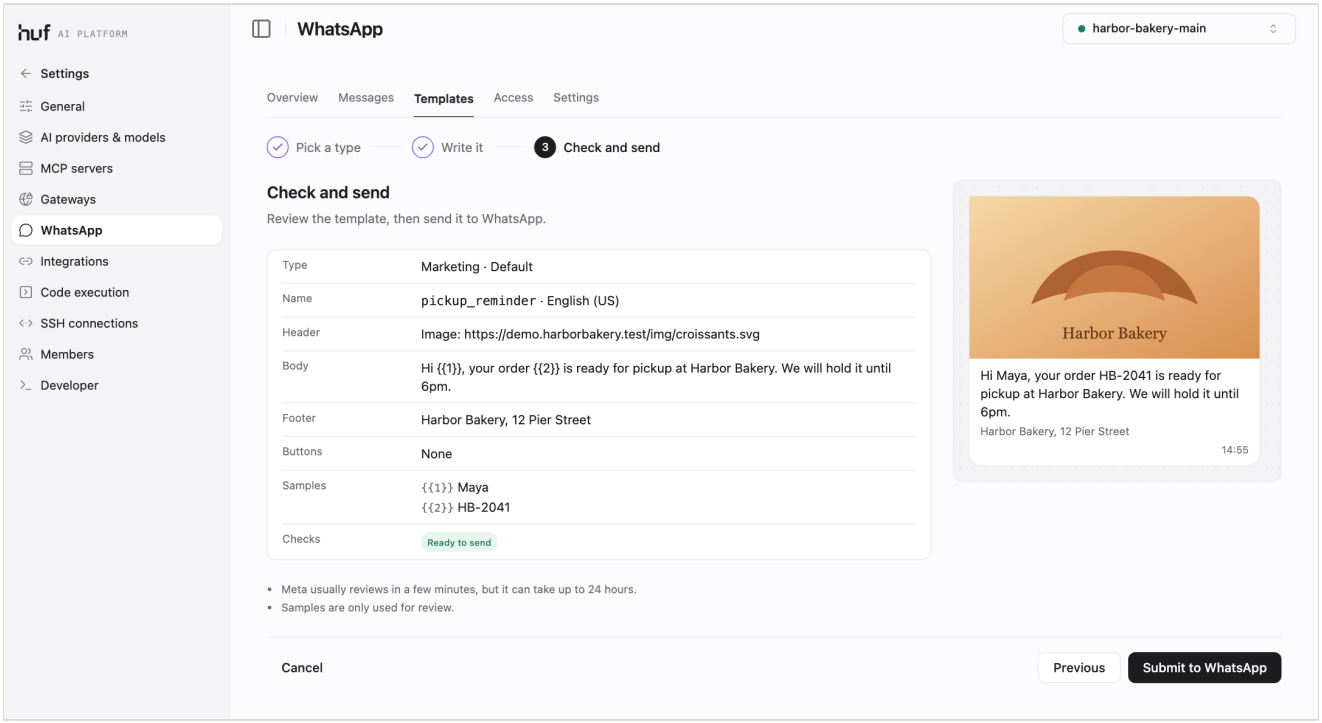


Figure 17. Step 3: check and send. Submit to WhatsApp. Screenshot of the Huf console, fictional Harbor Bakery data.

Rules enforced before Meta sees it

Part	Rule
Name	Lowercase letters, digits, underscore.
Body	Up to 1,024 characters; variables {{1}}, {{2}}... in order with no gaps; sample values required.
Header	TEXT (at most one variable, up to 60 chars), IMAGE or DOCUMENT. VIDEO headers are rejected.
Footer	Up to 60 characters.
Buttons	Quick replies (max 3), URL (static, or dynamic with {{1}} and an example URL), phone number; button text up to 25 characters.

All problems are returned together as one validation result, so the author fixes everything in one pass.

Sending with parameters

Templates can be sent with body parameters, header parameters (text, or an image or document link) and button parameters (dynamic URL suffixes). Empty parameters, or ones containing newlines, tabs or four or more spaces, are rejected locally because Meta rejects them. Templates are allowed outside the 24-hour window and are checked as Approved, against the contact's opt-out and against Meta's marketing stop.

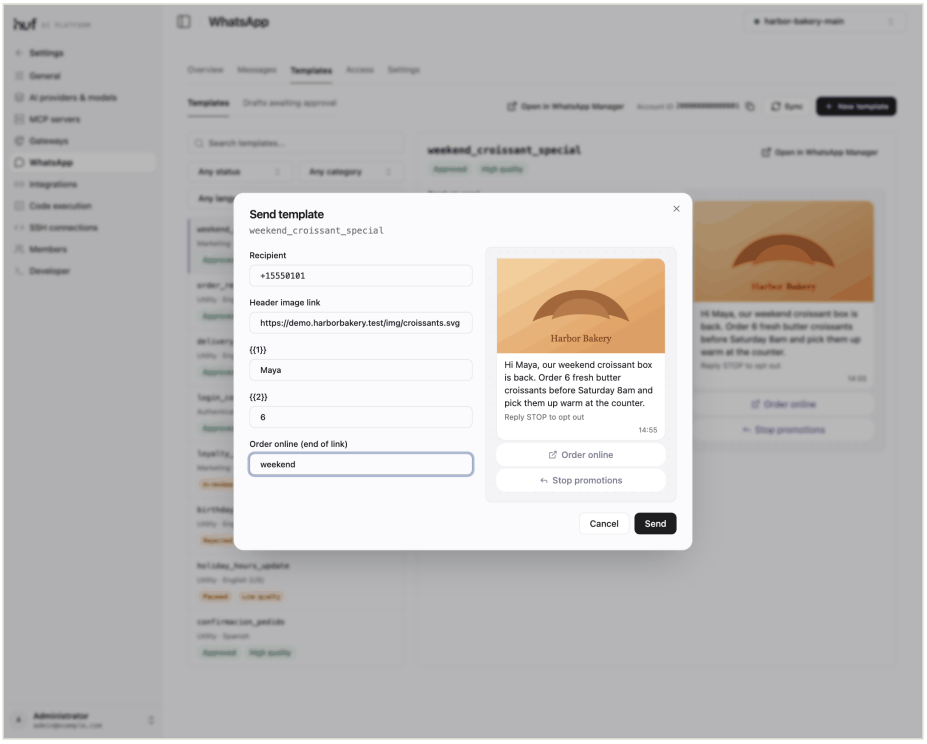


Figure 18. Send template dialog with recipient, header image, variables and a dynamic link filled in, and a preview. Screenshot of the Huf console, fictional Harbor Bakery data.

Status, quality and the edit quota

Huf keeps a template state record fed by Meta's webhooks (status, quality and category updates) and by Sync. It stores status (Approved, Pending, Rejected, Paused, Disabled, In appeal and more), the rejection reason and recommendation, quality (Green, Yellow, Red, Unknown), any pending category change and the last event. Each status change publishes a realtime event so the console updates live. Sync imports templates that exist on Meta but not in Huf and never pushes back.

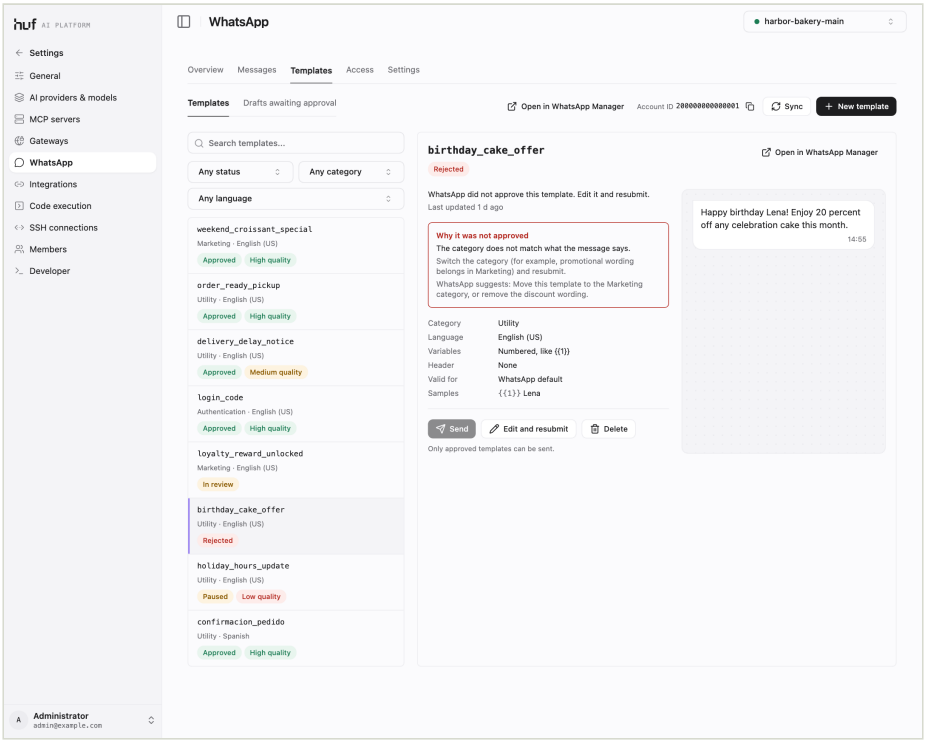


Figure 19. A rejected template, with the reason and advice in plain language. Screenshot of the Huf console, fictional Harbor Bakery data.

Edits follow Meta's rules, checked locally first: only Approved, Rejected or Paused templates can be edited; an Approved template once per 24 hours and 10 times per 30 days; name and language are fixed. Image and document header changes are not supported here.

The template guard

Huf-owned templates are protected from being re-pushed or deleted on Meta by an accidental save or delete in the underlying Desk, which would drop rich parts. The guard's message: This template is managed by Huf. Edit it from the WhatsApp page. A template that was already sent cannot be deleted (template_in_use).

> **AVOID ONE BUTTON**
>
> Do not use the connector app's own Fetch templates button; it bypasses the guard.

Drafts and human approval

Non-admins, and the agent, save a Template Draft. Nothing reaches Meta until a System Manager or Huf Admin approves. Approval re-validates and submits exactly once. A refusal from Meta marks the draft Failed so it can be retried or rejected.

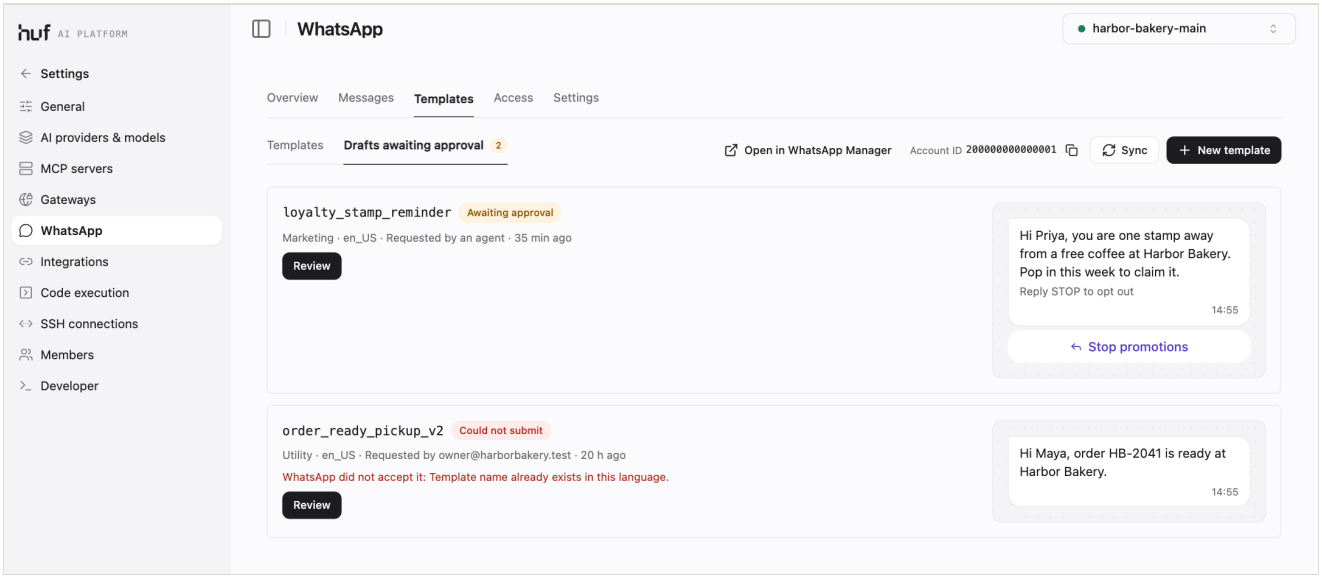


Figure 20. Drafts awaiting approval: an agent request and a failed submit. Screenshot of the Huf console, fictional Harbor Bakery data.

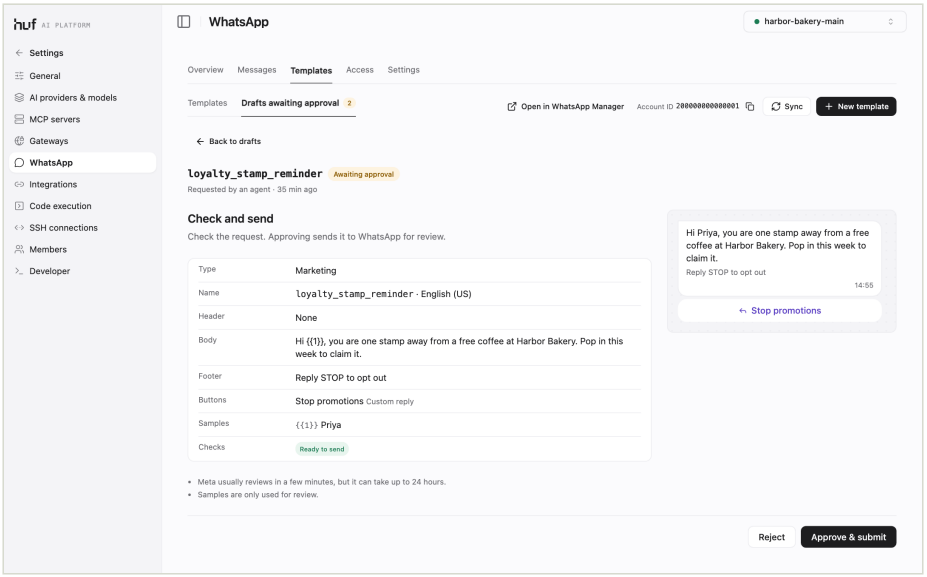


Figure 21. Reviewing a draft with a preview, Reject and Approve and submit. Screenshot of the Huf console, fictional Harbor Bakery data.

CONNECTOR REQUIRED

The Templates and Messages tabs need the optional frappe_whatsapp connector. Without it, the tab shows guidance to install it.

09

The 24-hour window and outbound policy

WhatsApp's rules, enforced for you, with a safe path when the window is closed.

Window rules

A free-form message is allowed only within 24 hours of the sender's last verified inbound message for the same Gateway. The window is anchored on the customer's own message timestamp (capped at receive time), so a Meta webhook redelivery cannot reopen it. Reactions and record-only kinds do not open it. It is computed from signed Gateway Events only.

> **CLICK-TO-WHATSAPP 72-HOUR WINDOWS**
> Meta's longer window for ad-originated conversations is not modelled. Huf conservatively applies 24 hours.

Fallback template

If a reply is due after the window closed:

- with an Approved fallback template configured, the template is sent instead and the event notes it;
- if the template is known not to be Approved, nothing is sent and the event fails with the template status;
- with no fallback, a clear window-closed failure is recorded with no retry loop.

Meta error 131047 on the first part also triggers the fallback. Parameters come from the configured list or the contact's name (or the word there). Health warns if Meta rejects, pauses or disables the fallback. Voice replies and pairing notices follow the same rule.

What an agent may send

Control	Default	Effect
Recipient scope	Current contact only	Can be widened to an allow-list of numbers, any Approved contact, or any number. Refusal: recipient_not_permitted.
Send caps	20 per run, 200 per rolling 24 h	Refusal: send_cap_exceeded with retry_after. Refused or failed sends do not count.
Proactive sends	Off	Free-form sends outside an open window need agent_allow_proactive. Templates are policy-checked instead.
Template management	Off	Agent template creation needs a flag and creates only a draft unless auto-submit is enabled.
Gateway binding	Automatic	The model never chooses the gateway or credentials; the tool is bound to the run's own gateway.

> **A PROMPT-INJECTED AGENT CANNOT SPAM OR REACH STRANGERS**
>
> Flags are read server-side from the run's Gateway; any such argument the model supplies is discarded, and a failure reading a flag fails closed. Credential-like arguments are discarded because tool arguments are persisted.

Idempotent sends

Each agent send claims a durable Send Key (the explicit idempotency_key, otherwise a hash of run, tool call, action and payload). A repeat returns the original result flagged deduplicated and sends nothing. A refused or failed send releases the claim; an unknown-outcome failure (timeout, 5xx) stays pending so a retry cannot double-send. Keys expire after 24 hours (15 minutes if pending).

Account state

When Meta reports the number Disconnected (partner removed, account offboarded or deleted, ban), every send is refused with gateway_disconnected. Restrictions and violations mark the gateway Degraded: sends are allowed and a warning shows. Reconnection restores Connected. A Gateway comment is kept as the audit trail.

10

Messages, delivery and drafts

A complete, searchable record of what was said and whether it arrived.

The message log

A gateway-scoped, newest-first log with filters for direction, status, agent, template, phone number, free text and date range. Each row shows who sent it (customer, agent, template or manual), a delivery-tick status and, on failure, a plain-language reason with the Meta code.

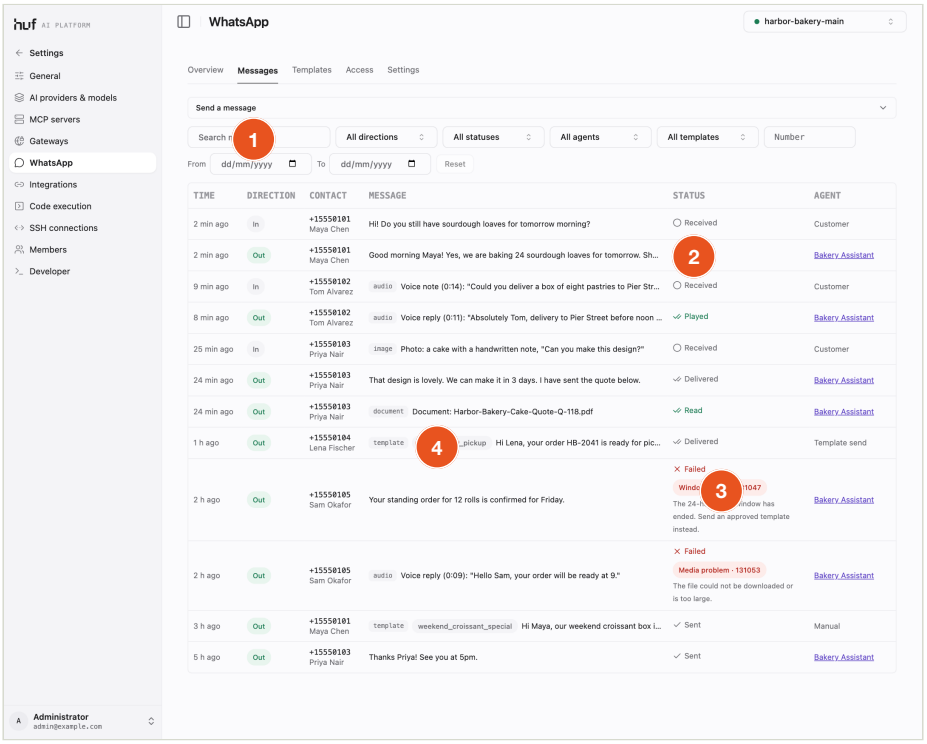


Figure 22. The Messages tab: text, voice note, image, document and template rows with delivery status and plain failure hints. Screenshot of the Huf console, fictional Harbor Bakery data.

1. Filters: direction, status, agent, template, number, dates.
2. Sent, delivered, read and played ticks.
3. A failed send with the code and a plain-language hint.
4. A template row, clearly labelled.

A failed status never downgrades a message that was already delivered or read, and a late sent never resurrects a failed one.

Detail sheet

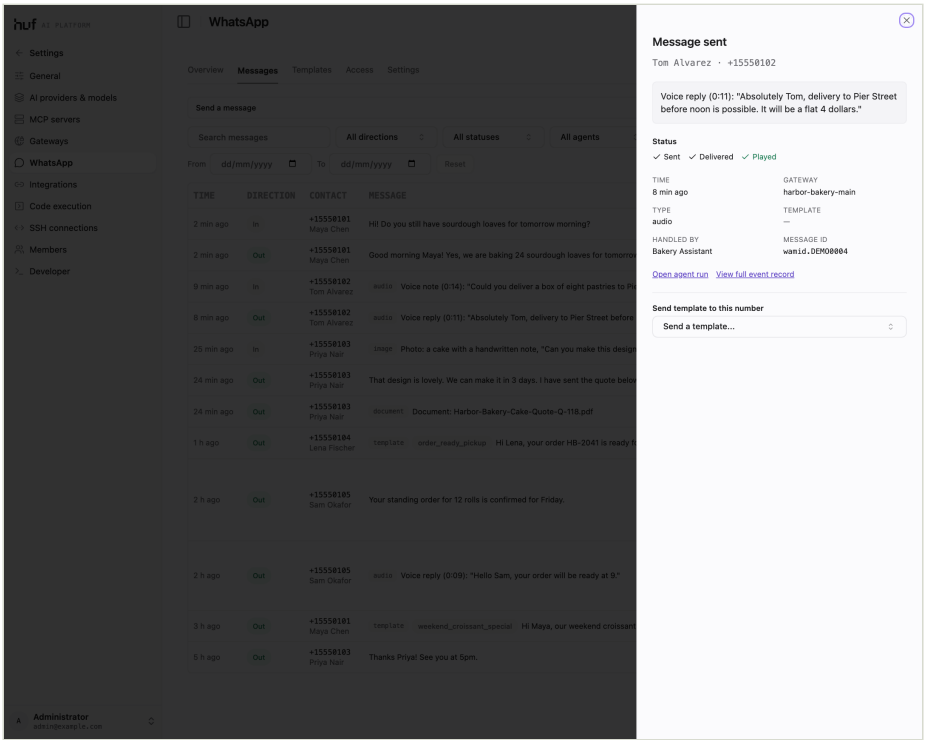


Figure 23. Detail sheet for a played voice reply: status timeline, handled by, link to the run. Screenshot of the Huf console, fictional Harbor Bakery data.

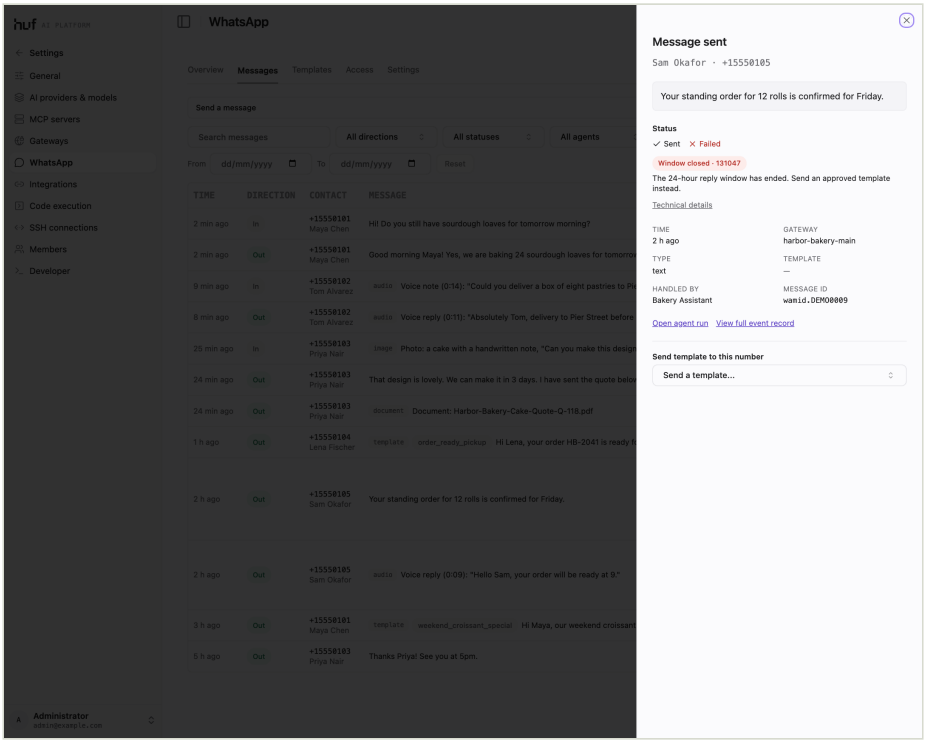


Figure 24. Detail sheet for a failed send: window closed (131047) with the hint to use a template. Screenshot of the Huf console, fictional Harbor Bakery data.

REPLY BODIES WITHOUT THE CONNECTOR

Without the connector the log is built from Gateway Events and reply bodies are not stored.

Events and observability

Every inbound is a Gateway Event with a status (Received, Recorded, Rejected, Unrouted, Queued, Running, Replying, Succeeded, Failed), a link to its Agent Run or Flow Run, the reply message id and its delivery status. Health is a pure function of database facts with five states (disabled, not set up, blocked, degraded, healthy) and reason codes each with a next step.

Health reason	In plain words
access_token_invalid	Meta rejected the token; replace it.
signature_failed	Meta is calling, but the signature check failed; check the app secret.
no_events_yet	Meta has not sent anything; check the callback URL and the messages subscription.
fallback_template_unavailable	The fallback template was rejected, paused or disabled.
voice_reply_ffmpeg_missing	Voice replies are on but ffmpeg is missing, so replies go out as text.
gateway_disconnected / degraded	Meta reported the account removed or restricted.

11

Agents, tools, flows and automations

Two ways to answer: an agent that reasons, or a flow that follows rules. Same number, same safety.

The agent's WhatsApp tool

One tool, whatsapp, with an action parameter. There are 20 actions:

Group	Actions
Send content	send_message, send_media, send_image, send_video, send_audio, send_voice_note, send_document, send_sticker
Send structure	send_location, send_contact, send_reaction, send_buttons, send_list, send_cta_url
Templates	send_template, list_templates, create_template, get_template_status
Read	list_messages (recent messages with this contact, limit 1 to 50), get_account_info

Everything is bound to the run's own gateway and subject to the recipient scope, caps and idempotency in chapter 9. Human-in-the-loop for templates is built in: agent-created templates are drafts for a person to approve.

> **DOCS VS CODE**
> Older setup notes list only seven actions and say non-text messages do not start a run. The code supports 20 actions, and location and contacts do start runs; this guide follows the code.

Routing to the right agent

Gateway Bindings are rules evaluated by priority, first enabled match wins: match any conversation, direct message, room, thread or exact sender, with an Agent or Flow as the target. No match falls back to the gateway default, else the event is Unrouted and no model is called. Agents can also orchestrate other agents, and flows offer router nodes.

> **ROUTER AGENTS ON WHATSAPP**
> A WhatsApp route to a router agent or router flow works through the normal Agent or Flow target, but no WhatsApp-specific router test exists, so the combination is not verified end to end.

Flows: deterministic on WhatsApp

There is no WhatsApp trigger node. You point a Gateway Binding, or the gateway default route, at a Flow. Huf then starts a Flow Run with trigger type Gateway, carrying the event, message, gateway, sender id and conversation id. The Flow's final output value becomes the reply; a flow with no output completes without replying; a failed flow fails the event. Duplicates start one run.

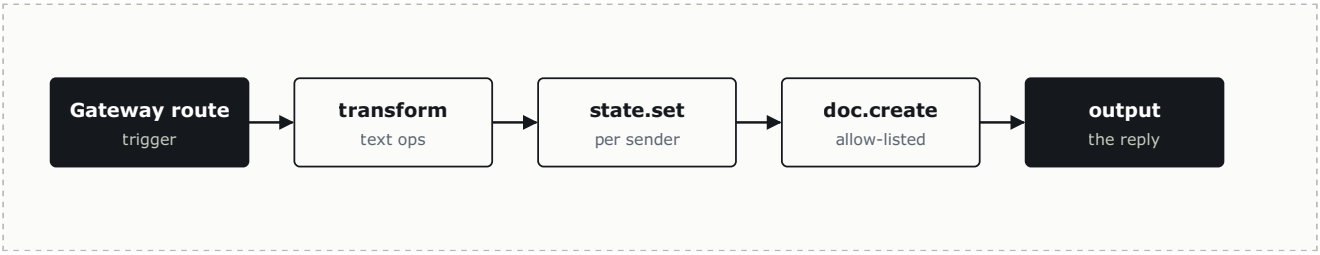


Figure 25. A Gateway-routed flow: text operations, per-sender state, a permission-checked document, and the reply. Illustration, not a screenshot.

- **Text ops (transform node, no LLM):** lower, upper, trim, replace, contains, starts_with, split_first, regex_extract (a conservative subset), concat; inputs bounded to 10,000 characters.
- **State:** state.get, set and delete scoped to the running flow, default TTL 24 hours (max 30 days), 16 KB per value, purged daily. Gateway runs carry the sender id, so a flow can remember waiting for an order number per sender.
- **doc.create:** creates a document only for doctypes in the site's allow-list (default: ToDo), as the run's user with normal permission checks (no ignore_permissions), plain data fields only, counted against the flow's max_writes limit.
- **Permission envelope:** each flow carries a permission envelope and limits (max_writes, max_external_calls and more).
- **Sending from a flow:** there is no dedicated send node. A tool.call node calls the whatsapp tool; the engine binds the gateway from the event that started the flow. A forged gateway name is ignored, and a flow without gateway context fails closed. The same recipient scope, caps and window rules apply.

Approvals

Any tool can be marked as requiring approval, and a flow author can place it behind a human.approval node. In flows, approvers are notified through the Notification Log and email, not over WhatsApp. Template drafts have their own human approval (chapter 8).

HONEST NOTE ON PROACTIVE SENDS AND AUTOMATIONS

The whatsapp tool works only from a gateway-bound run: an inbound chat, or a flow started by a Gateway route. Schedule and doc-event flows have no gateway context, and Automation-run agents cannot send WhatsApp messages with the tool. Reminders and ERP-event notifications today go through the console send panel, or the whitelisted send_template_message API called by an external script. Do not plan on a notification for every document event without that script.

NO GENERIC CUSTOMER APPROVAL PROMPTS AND NO HUMAN INBOX

Huf does not ask the customer to approve actions over WhatsApp, and there is no dedicated agent inbox or live takeover UI. A handoff is, for example, the agent filing a helpdesk ticket, plus a human replying from the console send panel.

12

Reliability and security

Built on the assumption that webhooks are redelivered, workers die, Meta rate-limits and callers lie.

Reliability

Mechanism	What it guarantees
Idempotency by message id	A unique hashed key per gateway and Meta message id; redelivery returns duplicate. The agent run also carries a client idempotency key so a retried event cannot execute twice.
Per-sender ordering	A lock per gateway and conversation serialises runs. A busy conversation is re-queued a bounded number of times, then failed with a fallback reply.
Outbound retries	Errors are classified (auth, rate limited, window closed, template, media, recipient, permission, parameter, server, timeout). Backoff 2, 4, 8, 16 seconds with jitter; fixed 6 seconds for Meta's pair rate limit; default 4 attempts. Permanent errors are not retried; Meta's marketing-limit error 131049 is never retried automatically and Huf waits 24 hours. If part 1 of a multi-part reply fails, part 2 is not sent, and a retry resumes with the unsent parts.
Reaper	Every 10 minutes: Running over 15 minutes fails, Queued over 10 re-enqueues, Received over 5 is replayed. Max 3 attempts and a 60-minute deadline, then Failed and the sender is notified.
Bounded redelivery	Payloads still unfinished after five 503 rounds are acknowledged and the remainder goes to dead letters: nothing is retried forever and nothing is lost silently.
Dead letters and replay	A deterministic ingest failure is kept as a Failed event with the redacted raw payload; an admin can replay it. Oversized deliveries ingest the first 500 messages then return 503 with Retry-After so Meta redelivers.

Failures are never silent, never leaky

A failed run sends a localised apology once per event (or the operator's own error text). An unopenable attachment gets its own message. Opted-out contacts get none. The agent can also return NO_REPLY to stay silent on purpose. Meanwhile the customer never sees internals: error text on events is redacted, payloads are scrubbed of keys such as token, secret and app_secret, and the token is stripped from Graph error text.

Security model

- HMAC signature and verify token with constant-time comparison; 3 MB body cap; per-gateway rate limits.
- Secrets in Password fields, redacted from logs, payloads and tool arguments.
- Least-privilege run-as user; no ignore_permissions in the agent tool or the management API.
- Gateway binding and privilege flags decided server-side.
- SSRF-checked media fetches, MIME allow-list and size caps on inbound media.
- STOP honoured before any send; recipient scope and send caps for agents.

- Forged traffic is still rate-limited after it fails the signature check: 100 requests per 60 seconds per gateway, 401 on a bad signature, 3 MB body cap (413 above it).
- Delivery status never goes backwards: a late sent cannot overwrite failed, and failed cannot overwrite read.
- Pairing is capped (3 requests per sender per day, 5 failed approvals per minute, 50 pending), so a sender cannot spam new pairing rows.

OPERATOR RISK TO KNOW

Meta allows one callback per app, and the optional frappe_whatsapp app ships an unsigned webhook route. Point Meta at Huf's URL and block the connector's route at the proxy. Anyone who knows the callback URL can make the gateway show blocked by sending a bad signature; this never affects message handling.

Tested against a strict simulator

There are about 1,070 WhatsApp tests (1,078 test functions in the WhatsApp test files). A local fake Graph server implements messages, templates, resumable upload and media endpoints and strictly validates every request body against Meta's documented schema: a schema violation fails the test. A call simulator covers the experimental calling feature.

SAY IT PRECISELY

This is tested against a strict Graph API simulator. It is not live-certified by Meta. The simulator does not model Meta's review decisions, rate limits or the 24-hour window, and live Meta has not been exercised for every shape.

13

Eight real use cases

Each is built from features in this guide. Conversations are illustrated with fictional people and +1 555 01xx numbers.

#	Use case	Highlights
1	Bakery order-taking and status	Reply buttons, formatting, run-as permissions
2	Appointment booking and reminders	List menus, templates, the 24-hour window
3	Support triage with pairing approval	Pairing codes, access approvals, ticketing
4	Internal ops bot	Flows, text ops, permission-checked doc creation
5	Field reporting	Photo, location and voice note into one record
6	Lead qualification from ads	Ad referrals, buttons, CRM lead creation
7	Collections and payment reminders	Templates with a dynamic button, STOP
8	Voice-first pharmacy	Speech-to-text, voice replies, multilingual

> **HOW TO READ THE SPREADS**
>
> Each use case takes two pages: the business and Huf setup on the first; the conversation, the behind-the-scenes steps and features showcased on the second.

USE CASE 1 OF 8

Bakery order-taking and order status

Rosa, owner of Maple Street Bakery, wants customers to order cakes by chat and ask where their order is.

Huf setup

- Gateway **Bakery Orders**, direct policy Open (or Allow list for a closed pilot).
- Run as user **bakery-bot** with read on Item and Sales Order and create on a custom Order Request doctype.
- Agent instructions: take cake orders, confirm size, flavour, pickup date and name, create an Order Request; answer status from Sales Order.
- Tools: ERP document tools and the whatsapp tool.
- Voice replies Off; fallback template **order_update** (no variables) for after-window pings; typing indicator on.

Features showcased

Reply buttons | Formatting | Run-as permissions | Idempotency | Typing indicator | Message log

WORTH KNOWING

Alternative: route the gateway to a Flow using state.set and doc.create into the allow-listed doctypes.

Gateway at a glance

The settings that define this use case.

Channel	**WhatsApp**
Direct messages	**Open (or Allow list for a pilot)**
Run as user	**bakery-bot**
Route	**Agent: order-taking agent**
Voice replies	**Off**
Fallback template	**order_update**
Typing indicator	**On**

Figure 26. Use case 1: the gateway's key settings, drawn as a card. Illustration, not a screenshot.

Use case 1: the conversation

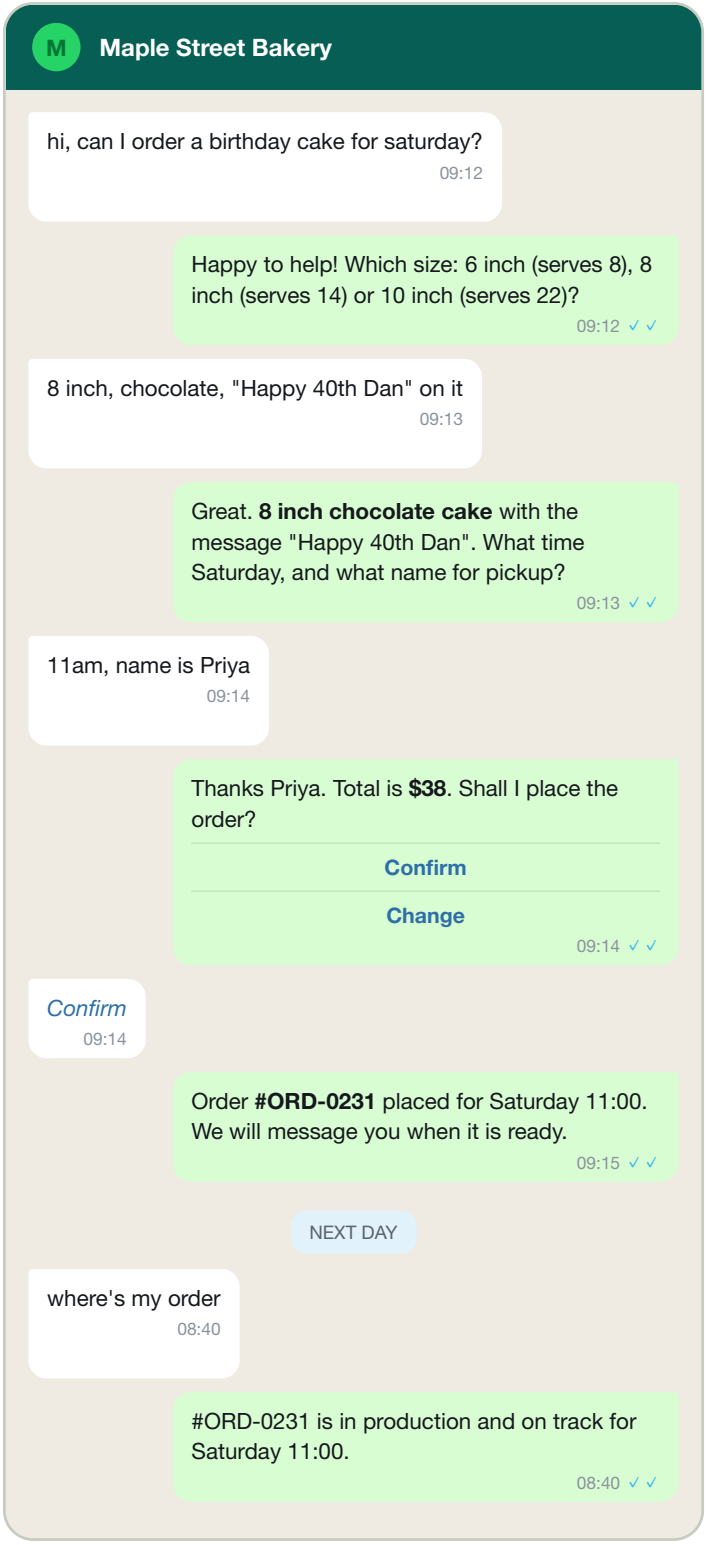


Figure 27. Use case 1: Bakery order-taking and order status. Illustrated conversation with fictional people and numbers; not a screenshot.

Behind the scenes

1. Webhook verified and de-duplicated; sender admitted; typing indicator shown.
2. The agent runs as bakery-bot under a per-conversation lock.
3. Markdown bold is converted to WhatsApp **bold**.
4. send_buttons offers Confirm or Change; the tap returns as an interactive reply whose title becomes the text.
5. The agent creates the order with the run user's permissions.
6. Next day's message finds the order via the stored sender; delivery ticks appear in the log.

Features showcased

Reply buttons · Formatting · Run-as permissions · Idempotency · Typing indicator · Message log

USE CASE 2 OF 8

Appointment booking and reminders

Dr. Lena Ortiz's dental clinic wants bookings by chat and reminders that arrive even when the 24-hour window has closed. Reminders go out as approved templates from the console or a script, not automatically on a schedule.

Huf setup

- Gateway **Clinic**: Allow list plus Pairing for new patients; recipient scope Approved contacts.
- Agent **Receptionist** with calendar and appointment tools.
- Template **appointment_reminder** (Utility): Hi {{1}}, your visit is on {{2}} at {{3}}. Reply 1 to confirm, 2 to cancel. With Confirm and Reschedule quick-reply buttons, created in the template studio and approved by Meta.
- Reminders are sent by an operator in the console send panel or by a script calling send_template_message, because scheduled flows cannot send WhatsApp today.

Features showcased

List menus | Template variables and buttons | 24-hour window | STOP honoured | Template status tracking

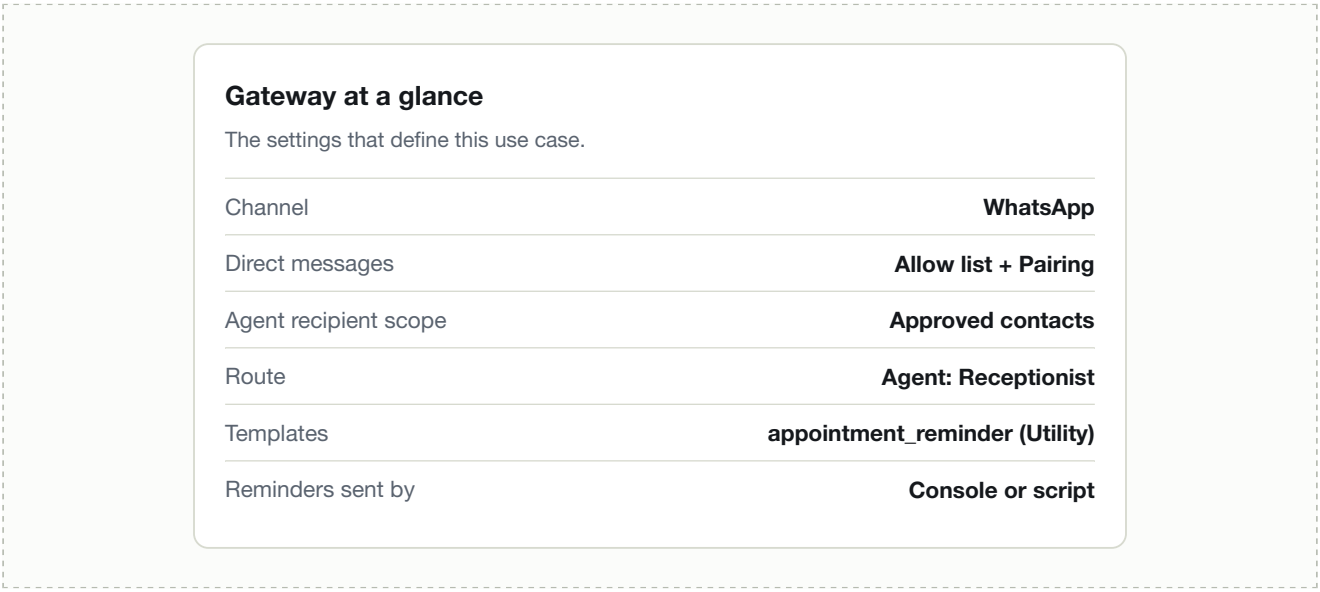


Figure 28. Use case 2: the gateway's key settings, drawn as a card. Illustration, not a screenshot.

Use case 2: the conversation

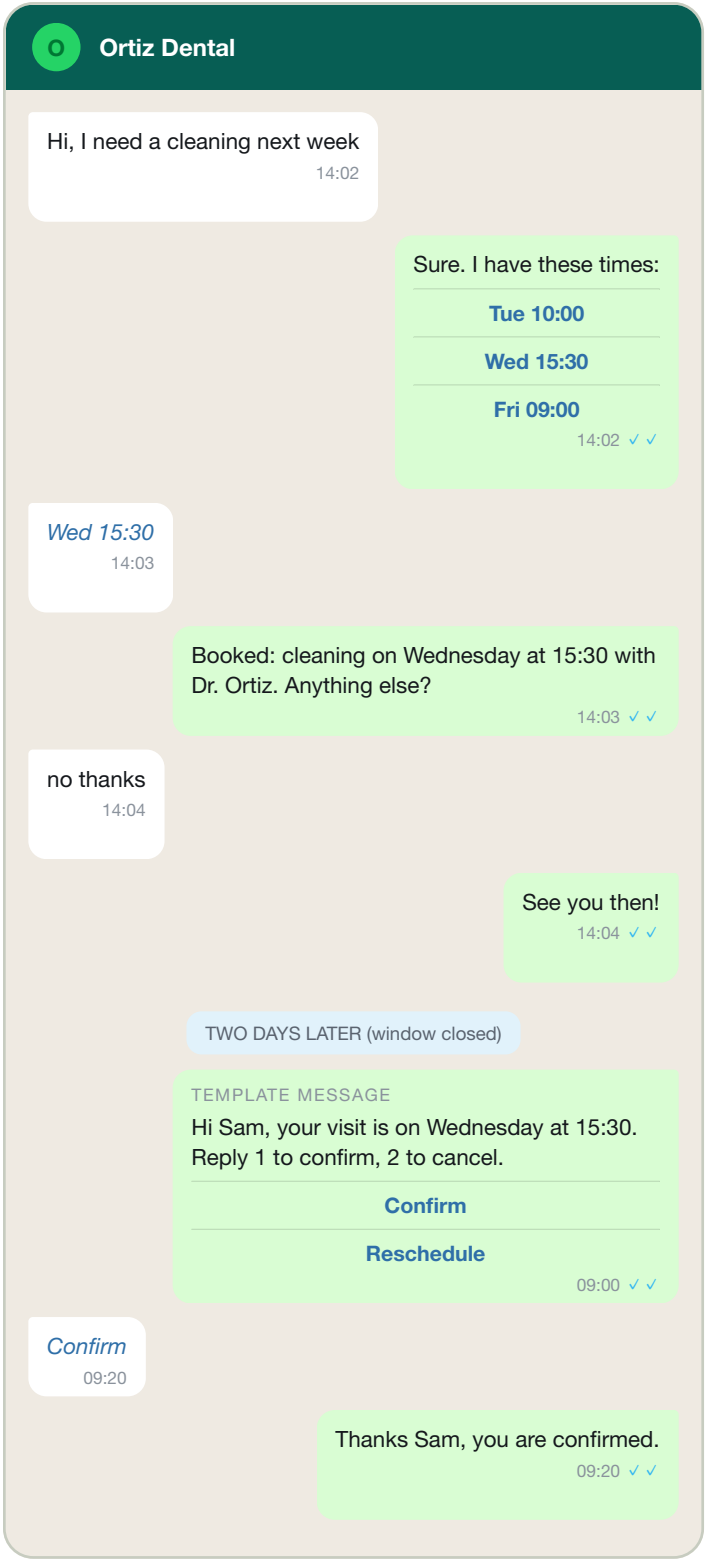


Figure 29. Use case 2: Appointment booking and reminders. Illustrated conversation with fictional people and numbers; not a screenshot.

Behind the scenes

1. The list menu is the agent's send_list action; the selection comes back as an interactive reply.
2. The template goes out through send_template or the management API, which is allowed outside the window.
3. It is checked as Approved and refused for a contact who sent STOP.
4. The customer's button tap reopens the 24-hour window, so the agent can reply free-form.

Features showcased

List menus · Template variables and buttons · 24-hour window · STOP honoured · Template status tracking

USE CASE 3 OF 8

Support triage with pairing approval and a ticket

NorthPeak Internet, a small ISP. Business customers get first-line AI help; unknown numbers must not.

Huf setup

- Gateway **NorthPeak Support**: direct policy Pairing, pairing reply enabled.
- Agent with helpdesk tools (requires the Helpdesk app), running as a low-privilege user.
- Staff approve new numbers in Console, Access. Approval admits the sender; it does not approve individual actions.
- Handoff: the agent files a Helpdesk ticket and says a human will reply; a person can then reply from the console send panel within 24 hours.

Features showcased

Pairing codes · Pending approvals · Per-sender limits · Run-as user · Fallback apology · No leakage

WORTH KNOWING

A dedicated WhatsApp agent inbox is not part of Huf today; the handoff is ticket creation.

Gateway at a glance

The settings that define this use case.

Channel	**WhatsApp**
Direct messages	**Pairing, pairing reply on**
Run as user	**Low-privilege support user**
Route	**Agent with helpdesk tools (Helpdesk app)**
Approvals	**Staff, in the Access tab**
Handoff	**Helpdesk ticket**

Figure 30. Use case 3: the gateway's key settings, drawn as a card. Illustration, not a screenshot.

Use case 3: the conversation

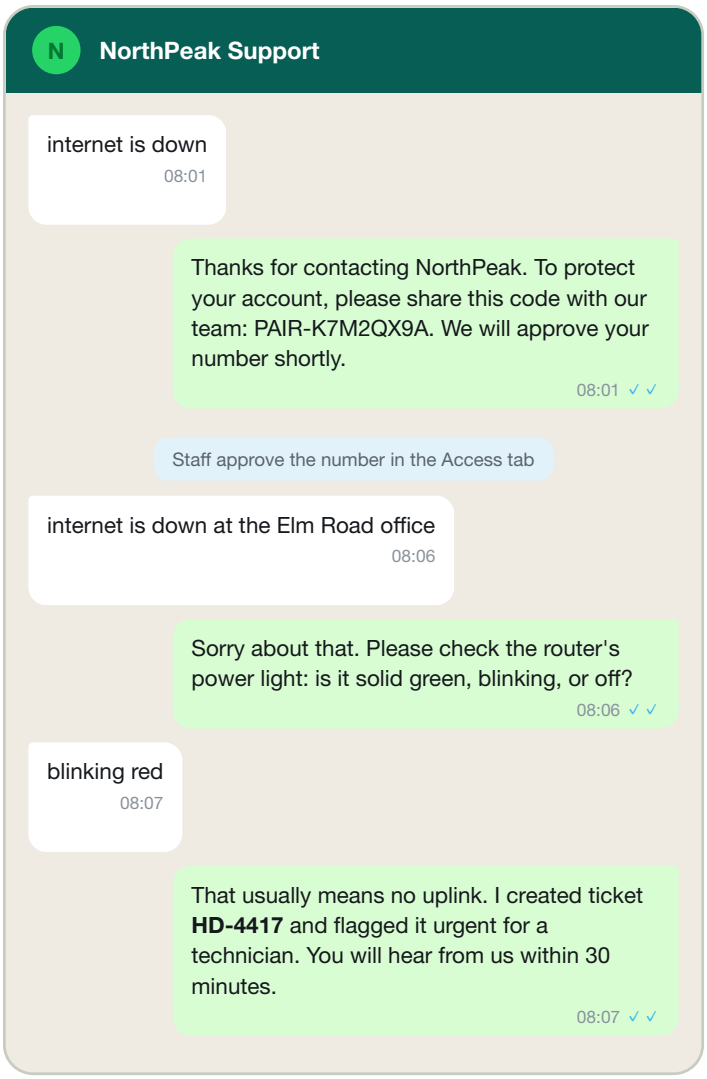


Figure 31. Use case 3: Support triage with pairing approval and a ticket. Illustrated conversation with fictional people and numbers; not a screenshot.

Behind the scenes

1. Pairing creates a pending Gateway Access Entry; the first message is not executed.
2. Approval restores admission.
3. The agent runs as a low-privilege user.
4. Unsupported content gets the fixed reply.
5. A failure produces the localised apology, never internals.

Features showcased

Pairing codes · Pending approvals · Per-sender limits · Run-as user · Fallback apology · No leakage

USE CASE 4 OF 8

Internal ops bot

Imran runs warehouse operations at a distributor; supervisors query stock and file tasks from their phones.

Huf setup

- Gateway **Ops Bot**: Allow list (supervisors only).
- Run as user **ops-bot** with read on Bin and Item and create on ToDo only.
- Flow **ops-intake** as the default route: transform text ops route by prefix (stock goes to a lookup agent via agent.run; task goes to doc.create ToDo), state.set remembers the last SKU per sender.
- Site allow-list for doc.create is set to ToDo.

Features showcased

Allow list | Flow as WhatsApp target | Text ops | doc.create under permissions | Per-sender state

WORTH KNOWING

Purchase requests can use a human.approval node; approvers are notified through the Notification Log and email, not WhatsApp.

Gateway at a glance

The settings that define this use case.

Channel	**WhatsApp**
Direct messages	**Allow list (supervisors)**
Run as user	**ops-bot (read Bin, Item; create ToDo)**
Route	**Flow: ops-intake**
doc.create allow-list	**ToDo**
Flow state	**Last SKU per sender**

Figure 32. Use case 4: the gateway's key settings, drawn as a card. Illustration, not a screenshot.

Use case 4: the conversation

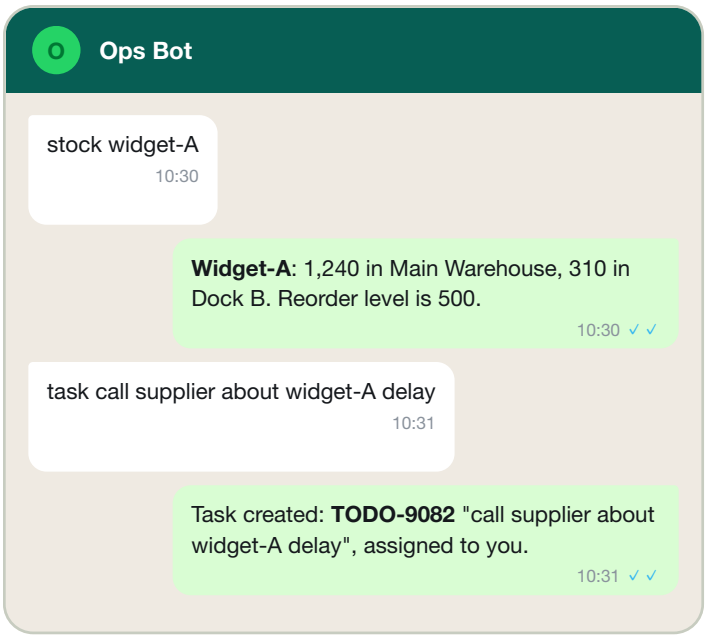


Figure 33. Use case 4: Internal ops bot. Illustrated conversation with fictional people and numbers; not a screenshot.

Behind the scenes

1. The allow list admits only supervisors.
2. Text ops are deterministic: no LLM decides the route.
3. doc.create inserts as the run user with permission checks and is limited by max_writes.
4. The flow's output value is the reply, in the same run.

Features showcased

Allow list · Flow as WhatsApp target · Text ops · doc.create under permissions · Per-sender state

USE CASE 5 OF 8

Field reporting: photo, location and voice note

A solar installer's technicians report site visits from the field.

Huf setup

- Gateway **Field Reports**: Allow list.
- Agent **Site Reporter** with a create-document tool for a custom Site Visit doctype.
- Speech-to-text configured on the agent (stt_model) and a vision-capable chat model.
- Optional: voice replies Text and voice.

Features showcased

Image vision | Location handling | Voice to text | Media size and MIME guards | Ordered processing

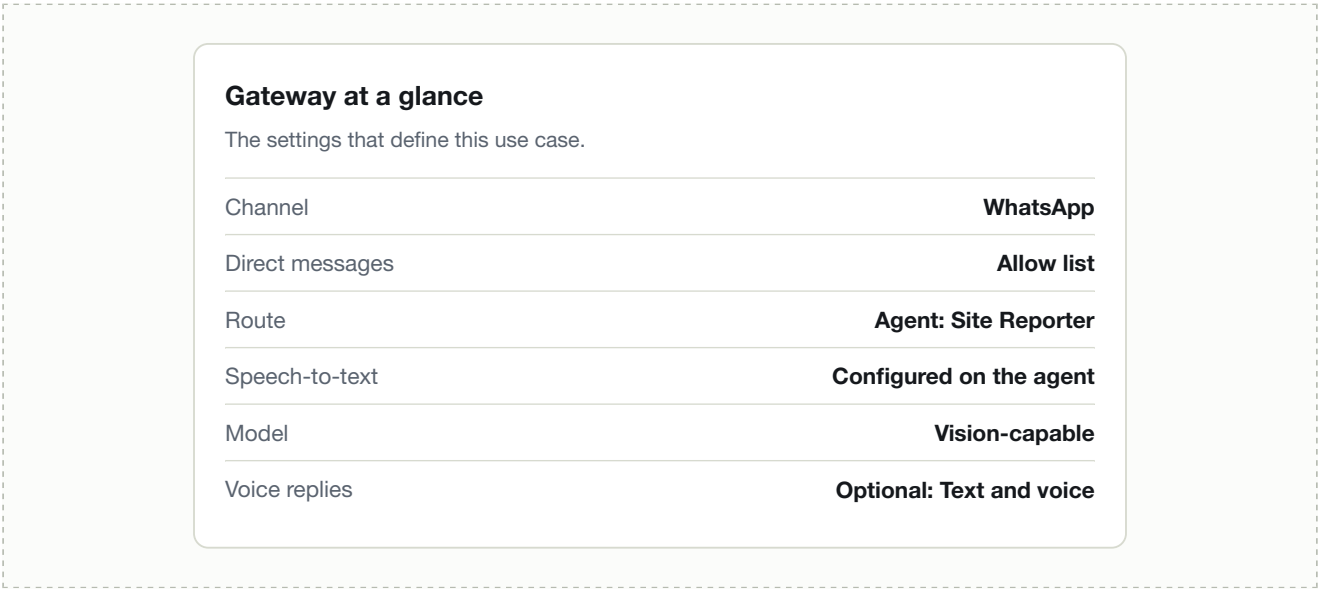


Figure 34. Use case 5: the gateway's key settings, drawn as a card. Illustration, not a screenshot.

Use case 5: the conversation

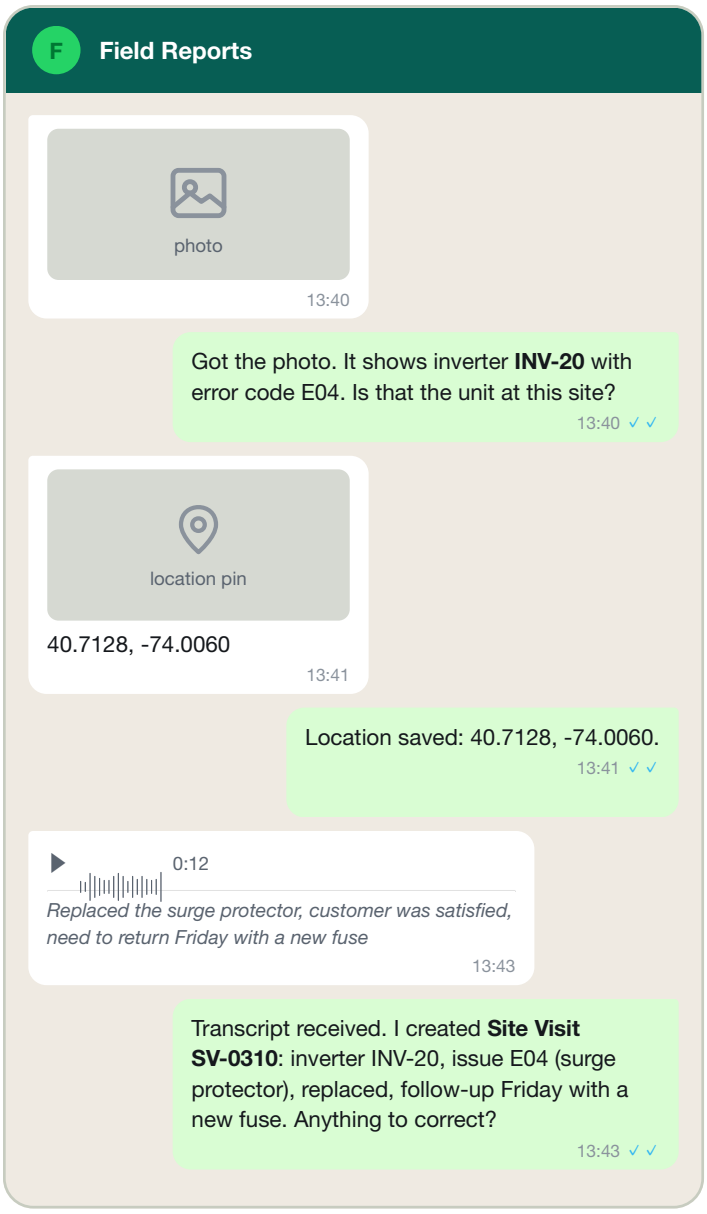


Figure 35. Use case 5: Field reporting: photo, location and voice note. Illustrated conversation with fictional people and numbers; not a screenshot.

Behind the scenes

1. The image reaches the model as vision input.
2. The location is rendered as [Location: ...] text.
3. The voice note is downloaded, transcribed and prefixed [Voice note transcript].
4. The agent files the record under the run user's permissions.
5. The original files stay attached to the Gateway Event; three quick messages are processed in order.

Features showcased

Image vision · Location handling · Voice to text · Media size and MIME guards · Ordered processing

USE CASE 6 OF 8

Lead qualification from ad clicks

A language school runs click-to-WhatsApp ads and wants leads qualified before an advisor calls.

Huf setup

- Gateway **School Leads**: direct policy Open, per-sender rate limit on.
- Agent qualifies course, level, start date and budget, and creates a CRM Lead with the CRM tool (requires the CRM app to be installed).
- Fallback template **follow_up_lead**, used when a reply would otherwise land outside the 24-hour window; later nurture messages are sent from the console or a script (nothing schedules them automatically).

Features showcased

Ad referral capture | Interactive buttons | Open policy with limits | STOP compliance | Fallback template

WORTH KNOWING

The free 72-hour entry window for ads is not modelled; Huf keeps 24 hours.

Gateway at a glance

The settings that define this use case.

Channel	**WhatsApp**
Direct messages	**Open**
Sender limit	**Default 20 per 10 minutes**
Route	**Agent with CRM tool (CRM app)**
Fallback template	**follow_up_lead**
Opt-out	**STOP / START defaults**

Figure 36. Use case 6: the gateway's key settings, drawn as a card. Illustration, not a screenshot.

Use case 6: the conversation

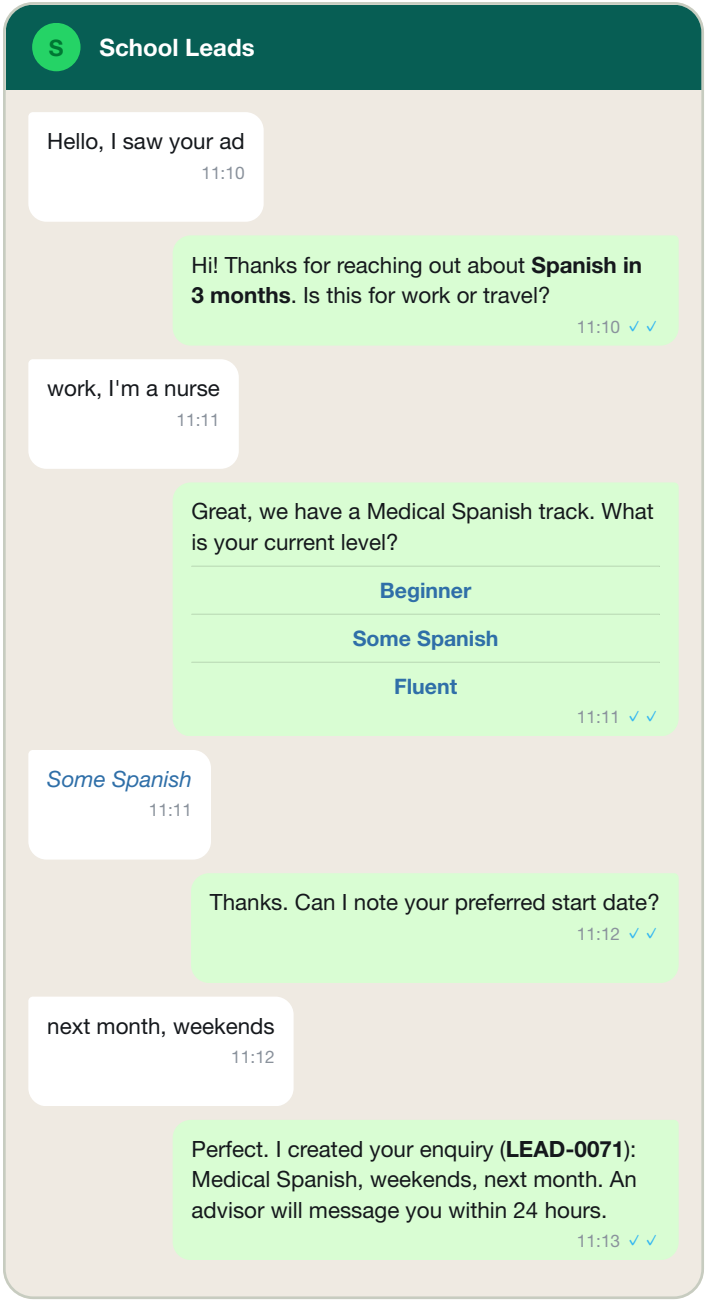


Figure 37. Use case 6: Lead qualification from ad clicks. Illustrated conversation with fictional people and numbers; not a screenshot.

Behind the scenes

1. The ad referral (headline, source id, ctwa_clid) is saved on the event.
2. The headline is prefixed to the first message as [Ad referral: ...] so the agent knows which ad.
3. The agent uses send_buttons for the level question.
4. The CRM tool creates the lead as the run user.

Features showcased

Ad referral capture · Interactive buttons · Open policy with limits · STOP compliance · Fallback template

USE CASE 7 OF 8

Collections and payment reminders

A tutoring centre sends fee reminders and handles parents' replies.

Huf setup

- Template **fee_reminder** (Utility): variables {{1}} student, {{2}} amount, {{3}} due date, and a dynamic URL button to the payment page (button_params).
- Reminder sent by a finance operator from the console send panel, or by a server script calling send_template_message.
- Agent **Fees Assistant** with read access to Fees handles replies.

Features showcased

Template variables and dynamic button

Window opened by a reply

STOP and START

Message log with ticks

WORTH KNOWING

Send-key idempotency applies on the agent path; idempotency of the management API path is not verified.

Gateway at a glance

The settings that define this use case.

Channel	**WhatsApp**
Template	**fee_reminder (Utility)**
Button	**Dynamic URL: Pay now**
Reminders sent by	**Console or script**
Route	**Agent: Fees Assistant**
Opt-out	**STOP / START defaults**

Figure 38. Use case 7: the gateway's key settings, drawn as a card. Illustration, not a screenshot.

Use case 7: the conversation

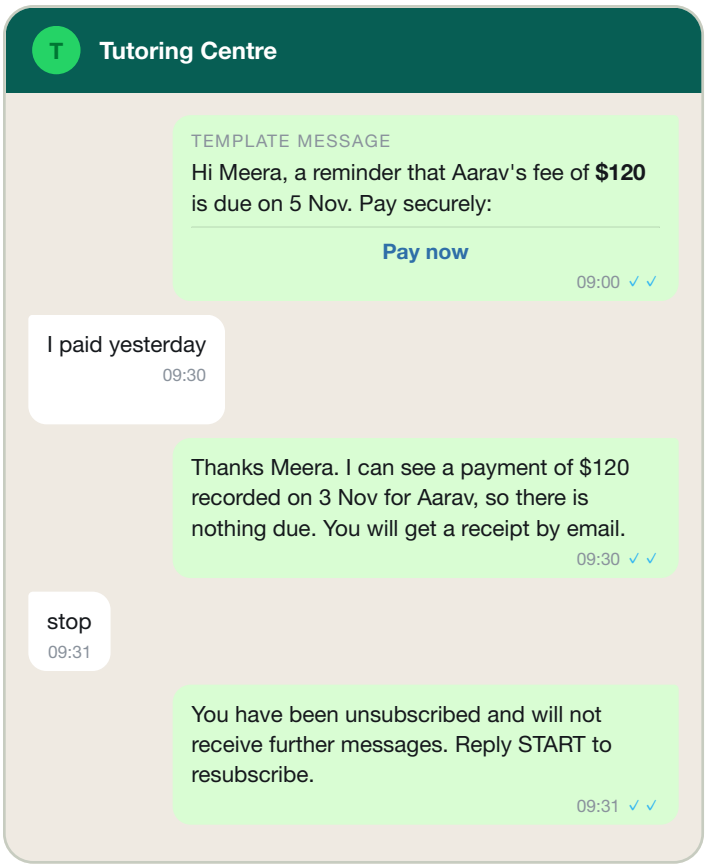


Figure 39. Use case 7: Collections and payment reminders. Illustrated conversation with fictional people and numbers; not a screenshot.

Behind the scenes

1. The template send is checked as Approved, against opt-out and Meta marketing stop.
2. The dynamic URL suffix is sent as a button parameter.
3. The parent's reply opens a 24-hour window so the agent can answer free-form.
4. STOP records the opt-out and blocks future sends, including templates.

Features showcased

- Template variables and dynamic button
- Window opened by a reply
- STOP and START
- Message log with ticks

USE CASE 8 OF 8

Voice-first customer in another language

A neighbourhood pharmacy serving customers who prefer Spanish voice notes.

Huf setup

- Gateway **Farmacia**: voice reply mode Voice only.
- Agent instructions: reply in the customer's language, short answers.
- Agent TTS model and voice and STT model configured.
- ffmpeg with libopus installed on every worker; health shows no voice_reply_ffmpeg_missing.

Features showcased

Speech-to-text | Voice replies with own key | Reply modes | Fallback to text | Localised apologies

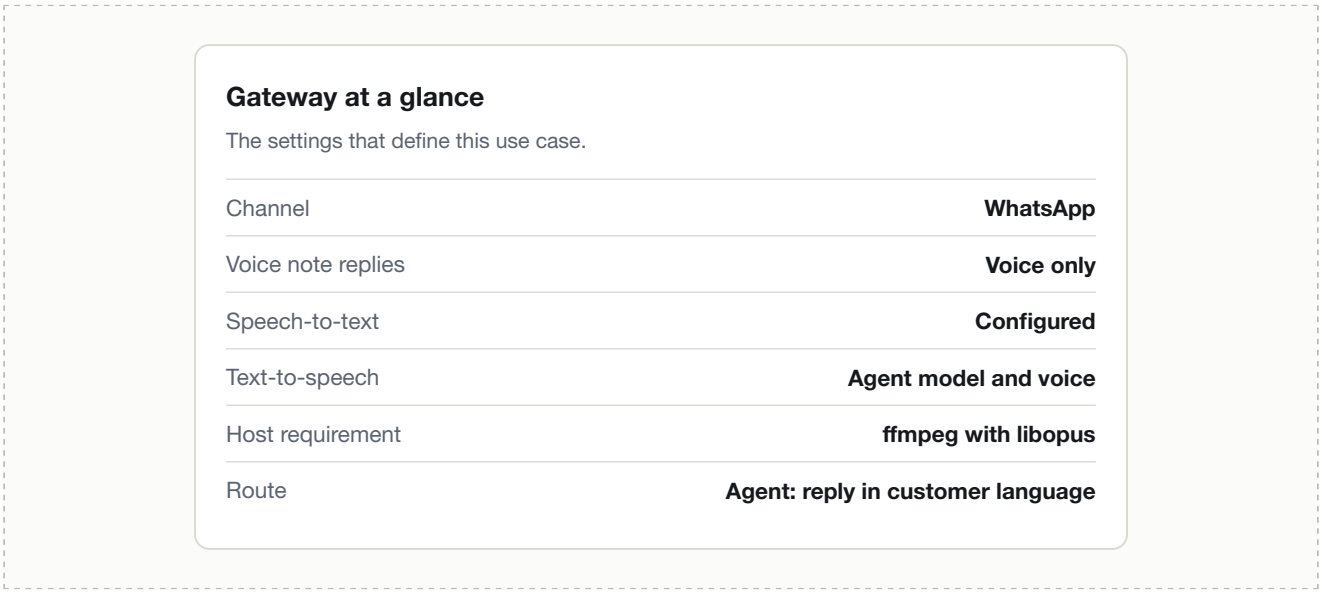


Figure 40. Use case 8: the gateway's key settings, drawn as a card. Illustration, not a screenshot.

Use case 8: the conversation

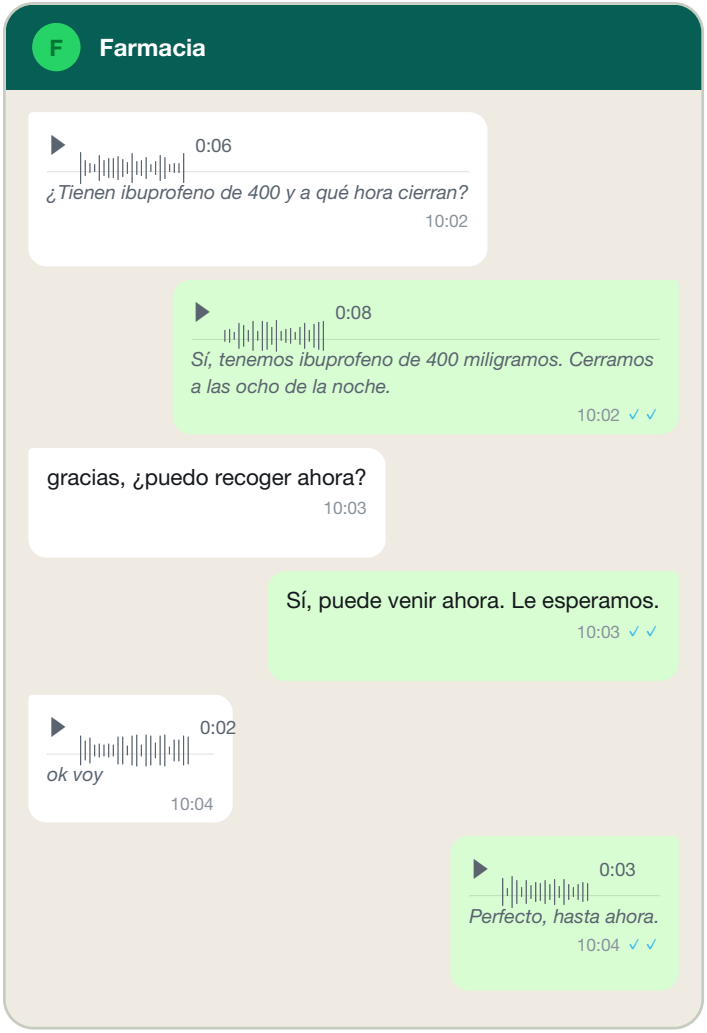


Figure 41. Use case 8: Voice-first customer in another language. Illustrated conversation with fictional people and numbers; not a screenshot.

Behind the scenes

1. The inbound audio is transcribed.
2. The agent answers in text; Huf synthesises it with the agent's TTS and converts it to Ogg/Opus.
3. Voice only replaces the text with one voice note; text messages get text.
4. If ffmpeg or TTS fails, the text reply is sent instead.

Features showcased

Speech-to-text · Voice replies with own key · Reply modes · Fallback to text · Localised apologies

14

Why Huf is a very strong WhatsApp solution

One agent platform that covers inbound understanding, safe outbound policy, reliability, and ERP/Frappe-grade permissions.

Most WhatsApp bots are a chat widget with a language model behind it. Huf starts from the other end: what must be true for a business to trust an AI with its WhatsApp number? Each point below is backed by code and tests.

Differentiator	What it means for you	Evidence
Verified, de-duplicated, never silently lost	HMAC signature before storage, hashed idempotency key, dead letters with replay, a reaper for stuck events, 503 + Retry-After for oversize.	test_gateway_no_lost_messages, test_gateway_no_silent_failure, test_whatsapp_ingest_guard
Compliance by construction	STOP/START before any agent runs; one outbound policy choke point; Meta marketing stop; 24-hour window anchored on the customer's timestamp.	outbound_policy.py, test_whatsapp_reply_pipeline
A prompt-injected agent cannot spam	Recipient scope, per-run and per-day caps, proactive flag off by default, gateway never model-controlled, durable idempotent sends.	send_scope.py, send_keys.py, test_whatsapp_agent_lockdown
Multi-number isolation	Each number is its own Gateway with its own window, log, state and signature.	test_whatsapp_multi_gateway_isolation
Handles messy WhatsApp content	Voice, photos, PDFs, location, contacts, reactions, edits, deletes, orders, ad referrals, Flow replies, unsupported types, BSUIDs.	whatsapp_inbound.py
Replies that look native	WhatsApp formatting, safe splitting with ordered delivery, typing indicator and read receipts, voice replies with text fallback.	whatsapp_format.py, voice_reply.py
Operator-grade console	Health with plain-language next steps, token expiry, checklist, delivery-tick log with timeline, template studio, access approvals.	frontend/src/components/whatsapp, health.py
First-class template lifecycle	Validation before Meta, drafts with approval, status and quality tracking, edit-quota awareness, protection from lossy re-pushes.	whatsapp_templates.py, whatsapp_template_guard.py
Least-privilege, auditable execution	Runs as a configured Frappe user, not Guest or Administrator; every inbound is an auditable event with run link and redacted payload.	gateway.py, gateway_service.py
Deterministic option beside the LLM	Route the same number to a Flow with text ops, per-sender state and permission-checked doc creation.	flow/create_doc.py, test_flow_whatsapp_gateway_trigger
Hardened against abuse and drift	Forged traffic is rate-limited after failing the signature check; delivery status never moves backwards; pairing requests are capped; marketing-limit errors are never auto-retried; unfinished oversize payloads end in dead letters.	gateway_webhook.py, gateway_service.py, whatsapp_errors.py

Differentiator	What it means for you	Evidence
Heavily tested	About 1,070 WhatsApp tests and a validating fake Graph server. Tested against a simulator, not live-certified.	huf/testing/fake_graph_server.py

Feature coverage matrix

Feature	Status	One-line proof
Signature verification	**In Huf today**	X-Hub-Signature-256 HMAC on every POST to an enabled gateway; 401 on a bad signature
Message de-duplication	**In Huf today**	Hashed per-message idempotency key; duplicate: true on redelivery
Voice note understanding	**In Huf today**	STT transcript prepended to the prompt
Image understanding	**In Huf today**	Vision input to the model
Document understanding	**In Huf today**	OCR and text extraction, size-capped
Location and contact cards in	**In Huf today**	Rendered as text; they start a run
Reactions, edits, deletes	**In Huf today**	Recorded; deleted unprocessed messages never answered
Formatted replies	**In Huf today**	Markdown to WhatsApp, 4,096-char split
Voice-note replies	**In Huf today**	Three modes, own TTS key, text fallback
Buttons and lists	**In Huf today**	Up to 3 buttons, up to 10 list rows
Media out (image, video, audio, document)	**In Huf today**	Link preflight or upload by id
Template studio	**In Huf today**	3-step wizard with live preview
Template drafts with approval	**In Huf today**	Draft, review, approve and submit once
Template status and quality tracking	**In Huf today**	Webhooks and Sync, realtime updates
24-hour window and fallback template	**In Huf today**	Window from the customer's timestamp
STOP / START	**In Huf today**	Editable keywords, enforced on every send
Pairing approvals	**In Huf today**	PAIR-XXXXXXXX codes, Waiting / Approved / Revoked
Run-as user permissions	**In Huf today**	Validated on save; Frappe permissions apply
Agent send lockdown	**In Huf today**	Scope, caps, proactive flag, idempotency keys
Multiple numbers	**In Huf today**	One Gateway per number, tested isolation
Delivery status ticks	**In Huf today**	Sent, delivered, read, played, failed with plain reasons
Retries and reaper	**In Huf today**	Backoff, classified errors, 10-minute reaper
Deterministic flows from WhatsApp	**In Huf today**	Gateway-routed flows with text ops, state, doc.create
Health and token checks	**In Huf today**	Banner, check chips, debug_token read

15

Honest limitations and roadmap

What Huf does not do yet, stated plainly so no claim is overstated.

Area	Where things stand
Embedded signup	Not available. The operator creates the Meta app, token, webhook and subscriptions manually; the checklist guides.
Coexistence	Using the WhatsApp Business app and the API on one number is not supported; no code was found.
Group chats	Not supported on WhatsApp. Group policy fields belong to other channels.
Template and history tabs	Need the optional frappe_whatsapp connector. In direct mode there are no Templates or Messages tabs and reply bodies are not stored.
Template types	Marketing and Utility can be created. Authentication (OTP) templates cannot be created here (synced ones show read-only). No carousel, catalogue, Flow or copy-code templates, and no VIDEO headers; image and document header edits are not supported.
Calling	Experimental and greeting-only: answers an inbound call, plays a pre-recorded clip, hangs up. Default off. No live conversation, no recording, no console UI. aiortc is not a declared dependency. Never run against Meta live in the evidence; business-initiated calls are unavailable for US, CA, EG, VN and NG numbers.
Proactive automation	The whatsapp tool works only from a gateway-bound run. Schedule and doc-event flows and Automations cannot send through it; use the console or send_template_message from an external script.
Human inbox	No dedicated agent inbox or live takeover. The console has a send panel.
Customer approval prompts	No generic approval prompt over WhatsApp. Flow approvals notify via Notification Log and email.
BSUID senders	Can be read but not answered: outbound needs a phone number.
72-hour ad window	Not modelled; 24 hours applies.
Forwarded and quoted context	Stored on the event, not shown to the agent.
Video understanding	Inbound video is saved as an attachment; content understanding is not confirmed.
Voice requirements	ffmpeg with libopus on every worker; health checks only the site host; replies over 1,500 characters go as text.
Account events	Phone-number change events are not handled by name.
Live Meta	Some response shapes (debug_token, subscribed_apps, user_preferences, some error codes, callback override requests) are not exercised live.
Router agents on WhatsApp	Not verified end to end.

WHY LIST THESE

A guide that only lists strengths is a brochure. These limits are what a careful buyer would find in an evaluation; the strengths in chapter 14 stand on their own evidence.

16

Appendix

Glossary

Term	Meaning
Gateway	One connected channel account, here one WhatsApp number, with its policies and route.
Gateway Event	The durable record of one inbound item and its outcome.
Run as user	The Frappe user whose permissions the agent uses.
Binding	A routing rule from a gateway context to an Agent or Flow.
Pairing	Admission where an unknown sender gets a code that staff approve.
24-hour window	Period after the customer's last message when free-form replies are allowed.
Fallback template	Approved template sent when a reply is due outside the window.
Template	Pre-approved message for business-initiated contact; Marketing or Utility here.
WABA	WhatsApp Business Account.
BSUID	Business-scoped user id, which Meta may supply instead of a phone number.
Bridge / direct mode	With or without the optional frappe_whatsapp connector.
Dead letter	A Failed event kept with its redacted payload for replay.
Reaper	The 10-minute job that rescues stuck events.
wamid	WhatsApp message id.
STT / TTS	Speech-to-text and text-to-speech.
Flow (Huf)	Deterministic graph workflow with nodes such as transform, state.*, doc.create, human.approval.

Meta error codes Huf explains in plain language

Code	Plain-language hint	Retried?
190	The access token is invalid or expired. Generate a new system user token and update the account.	No
131047	More than 24 hours since the customer's last message. Send an approved template instead.	No (falls back to template)
131056	Too many messages to the same recipient too fast. Wait a few seconds; other recipients are unaffected.	Yes, after 6 s
131049	Meta limited marketing messages to this recipient. Wait at least 24 hours.	No
131026	Undeliverable: number not on WhatsApp, terms not accepted, or old client.	No

Code	Plain-language hint	Retried?
131048	Blocked because the sender was flagged for spam. Improve quality and wait.	No
131031	Business account restricted or two-step PIN wrong. Resolve in WhatsApp Manager.	No
133010	The sending number is not registered with the Cloud API.	No
131053	Media upload failed or MIME type unsupported. Check file type and size.	No
131052	Inbound media could not be downloaded (for example larger than 100 MB).	No
132000	Template parameter count does not match the definition.	No
132001	Template does not exist in this language or is not approved.	No
132007	Template content violates a WhatsApp policy. Edit and resubmit.	No
132015	Template is paused for low quality. Edit it or wait for Meta to re-enable it.	No
132016	Template is disabled permanently. Create a new template.	No
131060	Informational; no action needed. Unsupported inbound items get no reply for this code.	n/a

Server-side and overload errors (for example 131016, 131057) are retried with backoff.

> **PROVENANCE**
> In the code's own error table, some codes are marked verified and others as from memory (for example 131000, 131005, 131008, 131016, 131021, 131057). The table above lists codes marked verified, plus 131060 which the code marks unverified.

Setup checklist

1. Create a Meta developer app and add the WhatsApp product.
2. Create a permanent system-user token with WhatsApp permissions.
3. In Huf, create a Gateway (channel WhatsApp) and Integration Settings with the four credentials.
4. Set the run-as user and verify it holds an allowed role and agent.use.
5. Choose a direct-message policy (Allow list, Pairing or Open) and a default Agent or Flow.
6. Paste the callback URL and verify token into Meta; subscribe to the messages field.
7. Add the app secret so signatures verify.
8. Turn the number on and message it from your own phone.
9. Click Check now and confirm the banner reads Working.
10. If using voice replies, install ffmpeg with libopus on every worker host.
11. Block the connector's unsigned webhook route at the proxy if frappe_whatsapp is installed.
12. Optionally set a fallback template, send scope and caps.

Where the code lives

Area	Path (in the Huf repository)
WhatsApp adapter	huf/ai/gateway_adapters/whatsapp.py
Connector bridge	huf/ai/gateway_adapters/whatsapp_bridge.py
Inbound rendering	huf/ai/gateway_adapters/whatsapp_inbound.py
Formatting and splitting	huf/ai/gateway_adapters/whatsapp_format.py
Payload builders	huf/ai/gateway_adapters/whatsapp_messages.py
Template rules and guard	whatsapp_templates.py, whatsapp_template_guard.py
Error classification	huf/ai/gateway_adapters/whatsapp_errors.py
Gateway service (ingest, admission, routing, replies, reaper)	huf/ai/gateway_service.py
Webhook entry point	huf/ai/gateway_webhook.py
Management API	huf/ai/whatsapp/management.py
Agent tool	huf/ai/tools/whatsapp.py
Outbound policy, scope, send keys	huf/ai/whatsapp/outbound_policy.py, send_scope.py, send_keys.py
Voice replies and file sending	huf/ai/whatsapp/voice_reply.py, send_file.py
Health and token check	huf/ai/whatsapp/health.py, token_check.py
Account events	huf/ai/whatsapp/account_events.py
Media preparation and OCR	huf/ai/media_prep.py, huf/ai/ocr_engine.py
Flows	huf/ai/flow/ (engine, create_doc, state)
Console UI	frontend/src/components/whatsapp/
Tests and simulators	huf/ai/whatsapp/tests/, huf/testing/fake_graph_server.py, wa_call_sim.py
Operator docs	docs/gateways/whatsapp-runbook.md, docs/WHATSAPP_INTEGRATION_SETUP.md